

2026 년 전기 졸업과제 중간보고서

멀티 AI 에이전트 기반 클라우드 네이티브 애플리케이션 개발 과정 개선 지원 기술 연구



부산대학교 정보컴퓨터공학부

지도교수: 염근혁

분과명 : 데이터베이스/그래픽스 및 비전(B)

팀명 : EasyDep

202113111 김재우 (projw1024@pusan.ac.kr)

202155578 이상현 (ask0208ask@pusan.ac.kr)

202055625 황수환 (tnghks2866@pusan.ac.kr)

목차

1. 과제의 목표	3
1) 과제 배경	3
2) 과제 세부 목표	4
2. 요구사항 및 제약사항 분석에 대한 수정사항	5
1) 기존 요구사항 및 수정사항	5
2) 변경된 유스케이스 다이어그램	8
3. 설계 상세화 및 변경 내역	9
1) 시스템 설계 및 명세	9
2) 전체 구조도	29
4. 갱신된 과제 추진 계획	31
5. 구성원 별 개발 진척도	32
6. 과제 수행 내용 및 중간 결과	33
1) 요구사항 분석 에이전트	33
2) 시스템 설계 에이전트	38
7. 향후 진행 계획	42
1) 시스템 구현 에이전트	42
2) 테스트 에이전트	42
3) 시스템 배포 환경 구성	42

1. 과제의 목표

1) 과제 배경

최근 클라우드 환경을 활용하여 서비스를 배포하고 애플리케이션으로 구축하는 클라우드 네이티브 애플리케이션 개발 방법이 주목받고 있다. 클라우드 네이티브 애플리케이션 개발 과정은 클라우드 리소스(네트워크, 가상머신 등)를 배포한 후, 서비스가 기능하는데 필요한 프레임워크, 소스 코드 등을 탑재 및 실행해야 한다. 즉, 클라우드 네이티브 애플리케이션 개발자는 클라우드 리소스를 선택하고, 클라우드 리소스 간 연결 관계를 고려하여 서비스를 배포하는 등의 단일 서비스 환경을 고려한 서비스 개발 과정에 비해 복잡한 서비스 개발의 의사결정 과정을 가진다. 최근 개발자 입장에서 개발 부담 감소를 위해, LLM(Large Language Model)의 추론 능력을 바탕으로 개발 요구사항을 입력하여 클라우드 리소스 배포 및 목표 서비스 실행에 필요한 산출물을 제공해주는 AI 에이전트 기술이 활용되고 있다.

그러나 클라우드 네이티브 애플리케이션을 개발하는 과정(클라우드 리소스 배포-리소스 연결 및 서비스 기능 탑재-서비스 실행)에서 AI 에이전트는 다음과 같은 문제가 발생할 수 있다. 첫째, AI 에이전트가 사용자의 의도를 정확하게 반영하지 못하는 환각(Hallucination) 현상으로 인해 실제 사용자의 클라우드 네이티브 애플리케이션 요구사항에 부합하지 않는 클라우드 리소스 환경과 서비스 기능을 산출물에 포함시킬 수 있다. 둘째, 클라우드 네이티브 애플리케이션은 클라우드 리소스 간 의존 관계성이 존재하며, 이에 따른 리소스 선택의 제약사항이 존재한다. 즉, 클라우드 리소스를 단순히 선택하는 것이 아닌, 특정 리소스를 선택하는 경우 연계되는 다양한 리소스 군을 획득할 수 있어야 하며, 사용자가 클라우드 리소스를 선택할 수 있는 가이드라인(선택된 클라우드 리소스의 비용, 성능 등) 제공이 필요하다. 셋째, 요구사항 분석, 설계, 구현, 테스트로 이어지는 소프트웨어 개발 과정을 고려하여 개발자의 클라우드 네이티브 애플리케이션 개발 과정을 지원하는 기술이 필요하다. 소프트웨어 개발 과정의 각 단계에서 도출되는 산출물을 일관된 기준에 따라 생성하며, 생성된 단계별 산출물이 요구사항에 부합하는지를 측정할 수 있는 기술이 마련되어야 한다.

따라서 본 과제에서는 클라우드 환경에 기반하여 클라우드 리소스 및 개발 단계별 산출물을 생성, 배포, 실행할 수 있는 멀티 AI 에이전트 기반 클라우드 네이티브 애플리케이션 개발 지원 기술을 제안한다. 제안하는 기술은 1) 요구사항 기반 클라우드 네이티브 애플리케이션 개발 산출물 검증 및 사용자 피드백을 위한 AI 에이전트 갱신 과정 설계, 2) 클라우드 환경 특성(클라우드 인스턴스 성능, 비용 등), 클라우드 리소스의 특성(리소스 용량, 리소스 의존성 등)을 고려한 클라우드 네이티브 환경 가이드라인 제공, 3) AI 에이전트 간 연계를 고려한 개발 단계별 산출물 생성 방법 구성을 목표로 한다.

2) 과제 세부 목표

- ① 요구사항 기반 클라우드 네이티브 애플리케이션 개발 산출물 검증 및 사용자 피드백을 위한 AI 에이전트 갱신 과정 설계
- ② 클라우드 환경 특성(클라우드 인스턴스 성능, 비용 등), 클라우드 리소스의 특성(리소스 용량, 리소스 의존성 등)을 고려한 클라우드 네이티브 환경 가이드라인 제공
- ③ AI 에이전트 간 연계를 고려한 개발 단계별 산출물 생성 방법 구성

2. 요구사항 및 제약사항 분석에 대한 수정사항

1) 기존 요구사항 및 수정사항

① 기능적 요구사항

표 1은 시스템이 제공해야 할 기능들에 대한 요구사항을 나타낸다. 굵게 표시된 부분은 추가된 요구사항이며, 밑줄 표시된 부분은 수정된 요구사항이고, 취소선으로 표시된 부분은 삭제된 요구사항이다.

표 1. 기능적 요구사항

기능		설명
요구사항 입력	클라우드 네이티브 애플리케이션 요구사항 입력	시스템은 개발하고자 하는 클라우드 네이티브 애플리케이션의 요구사항을 사용자에게 자연어 텍스트 형식으로 입력 받아 <u>요구사항 분석 에이전트로 전달한다.</u>
산출물 관리	요구사항 분석 산출물 도출	요구사항 분석 에이전트는 사용자의 입력을 바탕으로 <u>유스케이스 다이어그램, 유스케이스 명세 목록과 클라우드 리소스 명세를 생성하고 DBMS에 저장한다.</u>
	시스템 설계 산출물 도출	시스템 설계 에이전트는 <u>유스케이스 다이어그램, 유스케이스 명세 목록과 클라우드 리소스 명세를 바탕으로 시스템 설계 다이어그램(클래스, 시퀀스, ER, 배포) 및 API 명세서를 생성하고 DBMS에 저장한다.</u>
	<u>시스템 구현 산출물 도출</u>	<u>시스템 구현 에이전트는 시스템 설계 다이어그램 (클래스, 시퀀스, ER, 배포), API 명세서 및 클라우드 리소스 명세서를 기반으로 소스 코드 및 배포 파일을 생성하고 DBMS에 저장한다.</u>
	<u>테스팅 산출물 도출</u>	테스팅 에이전트는 생성된 <u>시스템 구현 산출물</u> 을 바탕으로 <u>테스트를 수행하여 테스트 결과를 도출한다.</u>
	사용자 피드백 분석	각 에이전트(요구사항 분석, 시스템 설계, 시스템 구현, 테스트)는 <u>사용자의 피드백을 분석하여 수정 범위를 파악할 수 있어야 한다.</u>
	산출물 재생성	시스템은 <u>수신된 피드백 내용에 따라</u> , 관련 에이전트를 통해 산출물을 수정하여 DBMS에 저장한다.

테스트 케이스 관리	테스트 케이스 생성	테스트 케이스 생성 에이전트는 각 단계별 산출물 이 생성될 때마다 이를 기반으로 요구사항을 검증 할 테스트 케이스를 생성하여 DBMS에 저장한다.
	테스트 케이스 검증	시스템은 테스트 결과에 따라 산출물을 재생성(테 스팅 결과 실패가 하나라도 포함)하거나 다음 개발 단계로 진행(모든 요구사항 부합)할 수 있어야 한 다.
	테스트 수행	시스템은 생성된 테스트 케이스들을 기반으로 테스 팅을 수행하고 테스트 성공 여부를 반환할 수 있어 야 한다.
애플리케이션 관리	앱 아이디 발급	시스템은 사용자에게 클라우드 네이티브 애플리케 이션의 식별자인 앱 아이디를 발급한다.
	클라우드 네이 티브 애플리케 이션 다운로드	사용자는 최종적으로 개발된 클라우드 네이티브 애 플리케이션의 산출물들을 파일 형태로 다운로드할 수 있어야 한다.

● 기능적 요구사항 수정 이유

기존에는 요구사항 분석 단계와 시스템 설계 단계에서 각각 테스트 케이스를 생성하고 테스트를 실행함으로써 요구사항의 누락 및 모순과 UML 다이어그램의 오류를 탐지한 후 보완하는 과정을 개발하고자 하였다. 그러나 구체적인 수행 방안을 검토한 결과, 본 과제에서 처리하는 자연어 요구사항이나 UML 다이어그램만으로는 테스트를 실제로 실행하고 성공 또는 실패를 판단하는 절차를 구성하는 데에 어려움이 있었다. 자연어 요구사항이나 UML 다이어그램을 실행 가능한 언어나 모델로 변환하여 검증하는 방법론은 존재하지만, 이를 적용하기 위해서는 자연어를 형식 명세로 변환하고, 다이어그램의 각 요소에 실행 의미를 부여하며, 정상과 오류를 판정하기 위한 조건을 별도로 정의해야 한다. 또한 별도의 테스트 도구를 시스템에 연동하고, 테스트 결과를 다시 원래 산출물의 수정 사항으로 연결하는 기능도 구현해야 한다.

이러한 과정은 기존의 개발 단계별 산출물 생성 기능에 테스트 도구를 단순히 추가하는 수준이 아니라, 별도의 형식 명세 및 모델을 테스트하는 체계를 설계하고 구현하는 작업에 해당한다. 또한 실행 가능하도록 변환된 산출물이 테스트를 통과하더라도, 변환 과정에서 사용자의 원래 의도가 잘못 반영되었다면 요구사항의 충족 여부를 보장하기 어렵다. 따라서 제한된 과제 범위에서 테스트 케이스 생성 및 실행 기능을 함께 구현하는 것보다, 각 개발 단계에서 필요한 산출물을 체계적으로 생성하고 사용자가 이를 직접 검토하여 의도를 반영하도록 하는 것이 본 과제의 목표에 더 적합하다고 판단하였다.

이에 따라 요구사항 분석과 시스템 설계 단계에서는 테스트 케이스를 통한 실행 기반 검증보다 산출물을 체계적으로 생성하고, 생성 결과를 사용자에게 제공하여 피드백을 통해 수정·보완하는 과정에 중점을 두도록 과제의 방향을 변경하였다. 테스트 케이스를 활용한 실

행 기반 테스트는 실제 실행 대상과 관찰 가능한 결과가 존재하는 시스템 구현 이후 단계에서 수행하도록 설계를 수정하였다.

② 비기능적 요구사항

표 2 는 시스템이 만족해야 할 비기능적 요구사항을 나타낸다. 밑줄 표시된 부분은 수정된 요구사항이다.

표 2. 비기능적 요구사항

요건	설명
가용성	시스템은 사용자가 필요로 할 때 언제든지 접근 및 활용이 가능해야 하며, 99.9% 이상의 가용성을 보장해야 함
신뢰성	<u>시스템은 물리적인 하드웨어 고장 및 오류로 인한 컨테이너 장애 발생 시 최대 5 분 이내에 파드의 컨테이너를 재시작하여 시스템을 복구할 수 있어야 함</u>
사용 편의성	시스템은 개발 지식이 부족한 기획자나 관리자도 대화형 인터페이스를 통해 시스템 요구사항을 쉽게 구체화할 수 있도록 UI/UX 를 제공해야 함
성능	시스템은 사용자가 클라우드 네이티브 애플리케이션의 개발 산출물의 제공을 요청할 때, 평균 5 초 이내에 제공할 수 있어야 함
유지보수성	<u>시스템은 소스 코드의 변경 사항 발생 시 CI/CD 도구(GitHub Actions)를 활용하여 빌드, 테스트, 배포 과정을 자동화할 수 있어야 함</u>

● 비기능적 요구사항 수정 이유

기존에는 유지보수성 항목에 신뢰성과 관련된 요구사항이 잘못 할당되어 있어서 해당 요구사항을 신뢰성 항목으로 이동하였다. 또한, 유지보수성 항목의 경우에는 시스템의 목표인고가용성과 신속한 장애 복구를 달성하기 위해서는 휴먼 에러(Human Error)를 최소화하고 상시 안정적인 배포 상태를 유지해야 하므로, 자동화된 테스트 및 배포 체계를 통해 시스템의 유지보수 효율성의 증가가 필요하다고 생각하여 해당 내용을 할당하였다.

2) 변경된 유스케이스 다이어그램

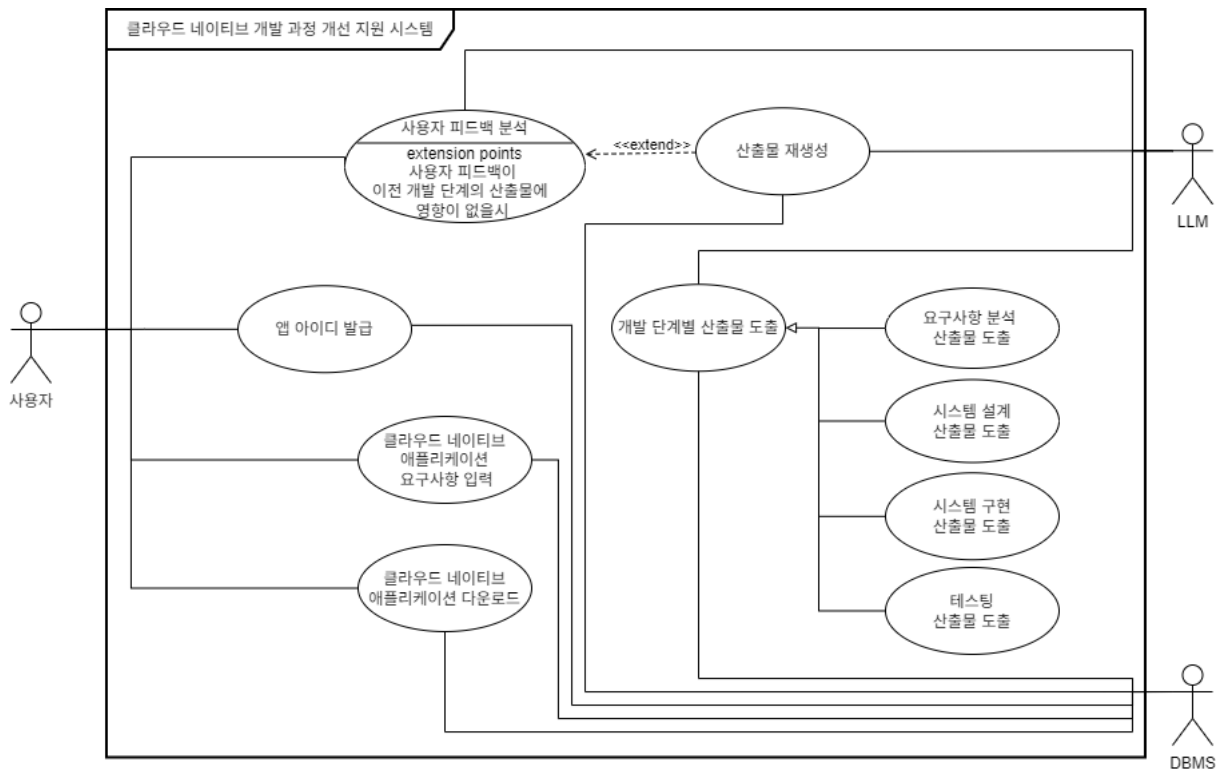


그림 1. 클라우드 네이티브 개발 과정 개선 지원 시스템 유스케이스 다이어그램

그림 1은 변경된 클라우드 네이티브 개발 과정 개선 지원 시스템 유스케이스 다이어그램이다. 기존 외부 액터로 배정되어있던 요구사항 분석 에이전트, 시스템 설계 에이전트, 시스템 구현 에이전트, 테스트 에이전트들은 구현중인 시스템에서 프롬프트와 내부 기능이 포함되어 있어 시스템 내부 요소에 해당하기 때문에 API를 통해 외부로부터 활용하는 LLM만을 외부 액터로 축소하였다. 그리고 시스템이 생성한 산출물을 사용자가 피드백하는 유스케이스를 추가하여 산출물의 품질을 높이도록 하였다. 또한 클라우드 네이티브 애플리케이션별 개발 산출물 및 개발 진행 상태를 식별하기 위한 앱 아이디를 발급하는 유스케이스를 추가하였다.

3. 설계 상세화 및 변경 내역

1) 시스템 설계 및 명세

① UML 클래스 다이어그램

● 애플리케이션 관리

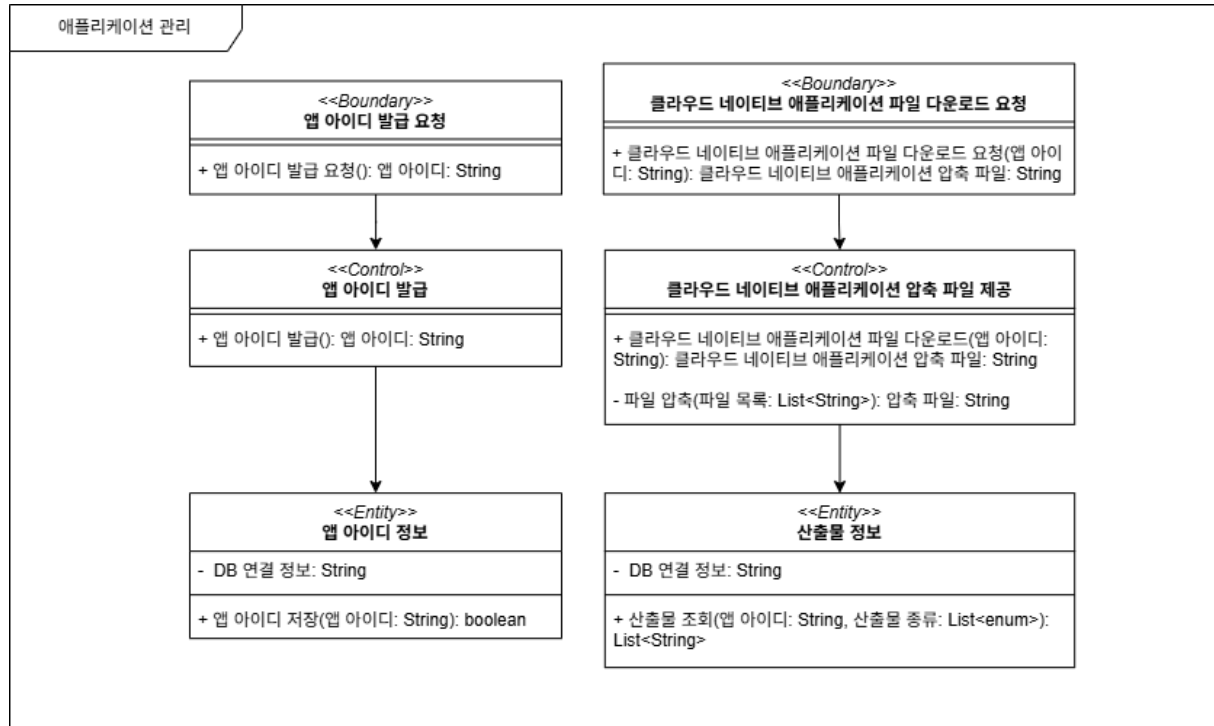


그림 2. 애플리케이션 관리 클래스 다이어그램

그림 2는 애플리케이션 관리에 필요한 구성 요소를 나타낸 클래스 다이어그램이다.

앱 아이디 발급 요청 클래스는 사용자가 시스템에 접속을 요청할 때 사용하는 클래스이다. 앱 아이디 발급 클래스는 앱 아이디 발급 요청 클래스로부터 사용자 접속 요청을 받아, 시스템 내에서 사용할 새로운 앱 아이디를 발급하는 작업을 수행한다. 앱 아이디 정보 클래스는 발급된 앱 아이디를 저장하는 역할을 담당한다.

클라우드 네이티브 애플리케이션 파일 다운로드 요청 클래스는 사용자가 클라우드 네이티브 애플리케이션 파일 다운로드를 요청할 때 앱 아이디를 입력받아 전달하는 인터페이스이다. 클라우드 네이티브 애플리케이션 파일 압축 파일 제공 클래스는 전달받은 정보를 바탕으로 조회된 파일 목록을 압축하여 최종적인 클라우드 네이티브 애플리케이션 압축 파일을 생성하는 로직을 수행한다.

● 요구사항 분석 산출물 도출

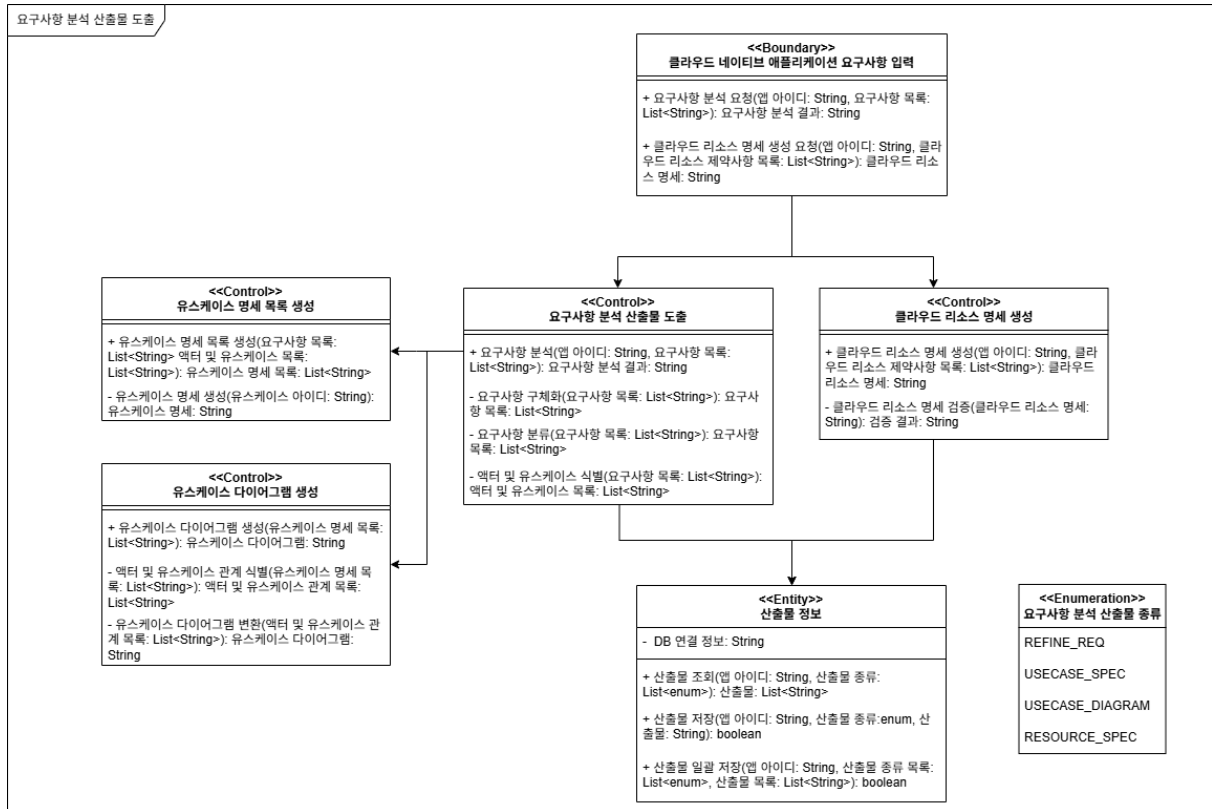


그림 3. 요구사항 분석 산출물 도출 클래스 다이어그램

그림 3 은 요구사항 분석 산출물 도출에 필요한 구성 요소를 나타낸 클래스 다이어그램이다.

요구사항 분석 산출물 종류 열거형(Enumeration) 클래스는 시스템이 진행하는 요구사항 분석 산출물들의 종류(REFINE_REQ, USECASE_SPEC, USECASE_DIAGRAM, RESOURCE_SPEC)를 정의하며, 클라우드 네이티브 애플리케이션 요구사항 입력 클래스는 사용자로부터 요구사항 목록 및 클라우드 리소스 제약사항 목록을 입력받아 전달하는 인터페이스이다.

요구사항 분석 산출물 도출 클래스는 요구사항 구체화 및 분류 과정을 거쳐 요구사항을 정제한 후, 해당 요구사항을 바탕으로 액터 및 유스케이스를 식별하는 로직을 수행한다. 유스케이스 명세 목록 생성 클래스는 식별된 액터 및 유스케이스를 바탕으로 각 유스케이스들에 대한 명세를 생성하고 검증하는 로직을 수행한다. 유스케이스 다이어그램 생성 클래스는 유스케이스 명세 목록을 바탕으로 액터 및 유스케이스 간의 관계를 식별 및 검증하여 다이어그램으로 변환하는 로직을 수행한다. 클라우드 리소스 명세 생성 클래스는 전달받은 클라우드 리소스 제약사항 목록과, 앞서 도출한 요구사항 목록 중 비기능적 요구사항 목록을 바탕으로 클라우드 리소스 명세를 생성하고 검증하는 로직을 수행한다.

산출물 정보 클래스는 클라우드 네이티브 애플리케이션을 식별하는 앱 아이디와 개발 산출물들의 종류를 기준으로 DBMS 에서 산출물들을 조회하거나 저장하는 역할을 수행한다.

● 시스템 설계 산출물 도출

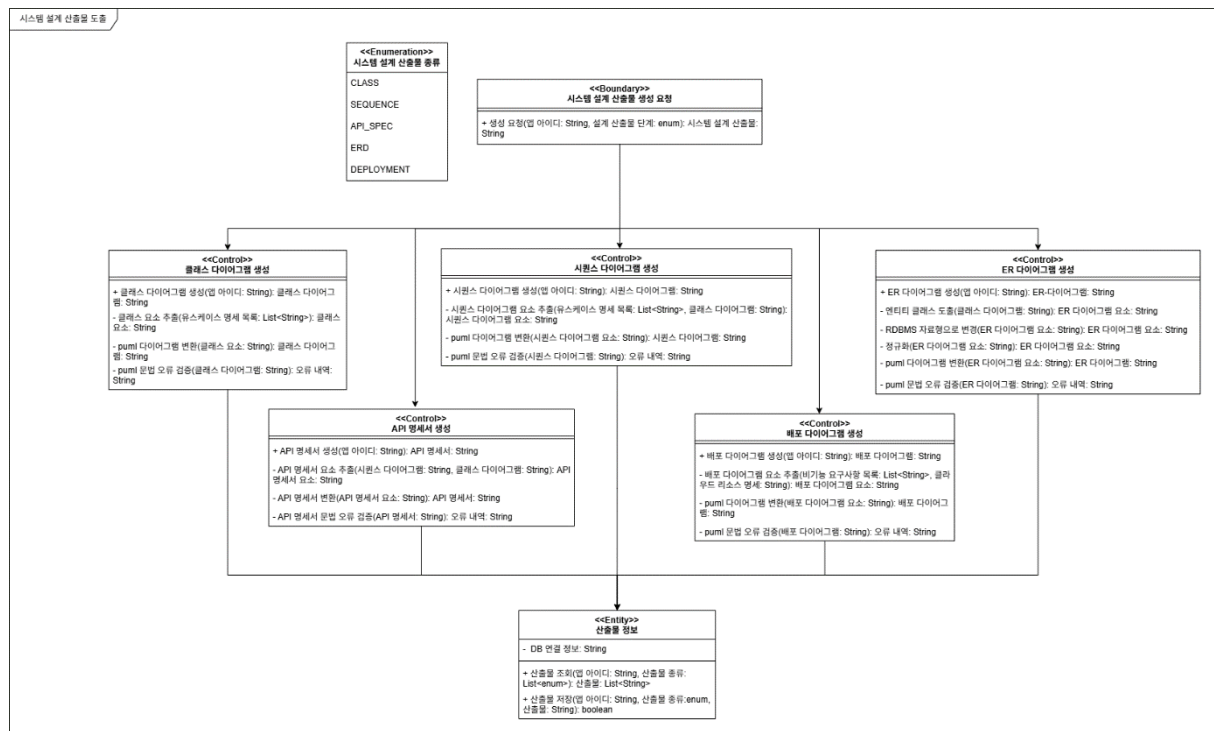


그림 4. 시스템 설계 산출물 도출 클래스 다이어그램

그림 4는 시스템 설계 산출물 도출에 필요한 구성 요소를 나타낸 클래스 다이어그램이다.

시스템 설계 산출물 종류 열거형 클래스는 시스템이 생성할 수 있는 설계 산출물의 종류 (CLASS, SEQUENCE, API_SPEC, ERD, DEPLOYMENT)를 정의하며, 시스템 설계 산출물 생성 요청 클래스는 사용자가 시스템 설계 산출물 생성을 요청할 때 앱 아이디와 설계 산출물 종류를 입력받아 전달하는 인터페이스이다.

클래스 다이어그램 생성 클래스는 유스케이스 명세 목록으로부터 Boundary-Control-Entity 스테레오 타입의 클래스 요소를 추출하여 다이어그램으로 변환하고 문법 오류를 검증하는 로직을 수행한다. 시퀀스 다이어그램 생성 클래스는 유스케이스 명세와 클래스 다이어그램으로부터 시퀀스 요소를 추출하여 다이어그램으로 변환하고 문법 오류를 검증하는 로직을 수행한다. API 명세서 생성 클래스는 시퀀스 다이어그램과 클래스 다이어그램으로부터 API 명세서 요소를 추출하여 변환하고 문법 오류를 검증하는 로직을 수행한다. ER 다이어그램 생성 클래스는 클래스 다이어그램을 바탕으로 엔티티 도출, RDBMS 자료형 변경 및 정규화 과정을 거쳐 다이어그램으로 변환하고 문법 오류를 검증하는 로직을 수행한다. 배포 다이어그램 생성 클래스는 비기능적 요구사항과 클라우드 리소스 명세로부터 배포 요소를 추출하여 다이어그램으로 변환하고 문법 오류를 검증하는 로직을 수행한다.

● 시스템 구현 산출물 도출

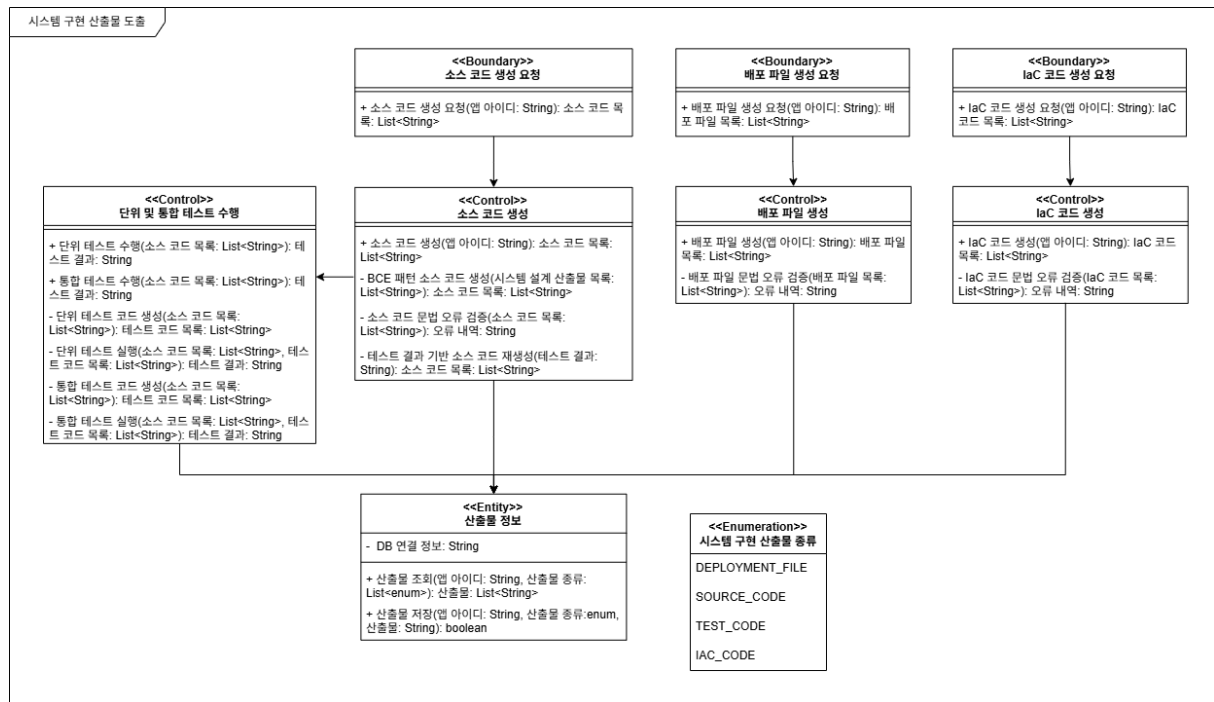


그림 5. 시스템 구현 산출물 도출 클래스 다이어그램

그림 5는 시스템 구현 산출물 도출에 필요한 구성 요소를 나타낸 클래스 다이어그램이다.

시스템 구현 산출물 종류 열거형 클래스는 시스템이 생성할 수 있는 구현 산출물의 종류 (DEPLOYMENT_FILE, SOURCE_CODE, TEST_CODE, IAC_CODE)를 정의하며, 소스 코드 생성 요청 클래스와 배포 파일 생성 요청 클래스는 사용자가 소스 코드 및 배포 파일 생성을 요청할 때 앱 아이디를 입력받아 전달하는 인터페이스이다. IaC 코드 생성 요청 클래스는 사용자가 IaC 코드 생성을 요청할 때 앱 아이디를 입력받아 전달하는 인터페이스이다.

소스 코드 생성 클래스는 전달받은 앱 아이디로 시스템 설계 산출물을 조회한 후 이를 바탕으로 Boundary-Control-Entity 패턴의 소스 코드를 생성 및 검증하고, 테스트 결과에 따라 코드를 재생성하는 로직을 수행한다. 배포 파일 생성 클래스는 배포 파일을 생성하고 문법 오류를 검증하는 로직을 수행한다. 단위 및 통합 테스트 수행 클래스는 생성된 소스 코드에 대한 단위 및 통합 테스트 코드를 생성하고 이를 직접 실행하는 로직을 수행한다. IaC 코드 생성 클래스는 IaC 코드를 생성하고 문법 오류를 검증하는 로직을 수행한다.

● 테스트 산출물 도출

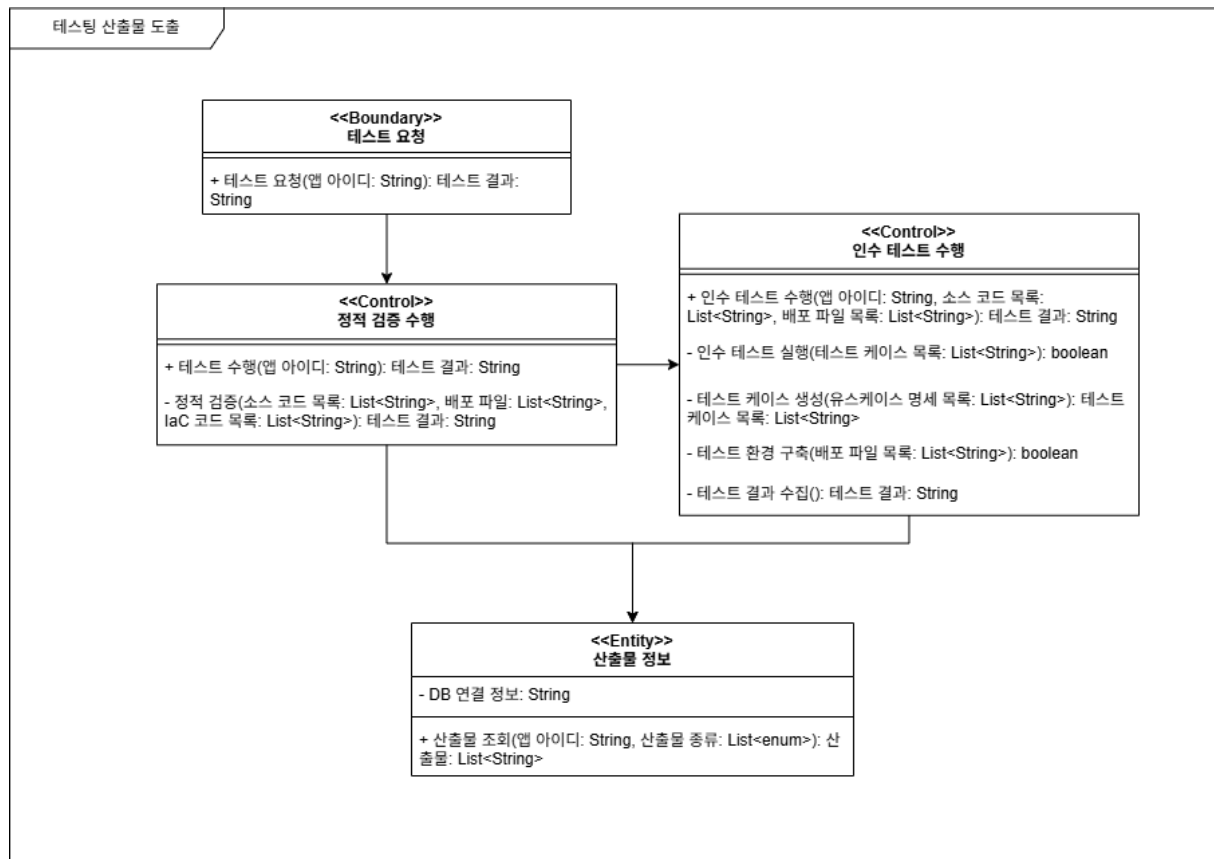


그림 6. 테스트 산출물 도출 클래스 다이어그램

그림 6은 테스트 산출물 도출에 필요한 구성 요소를 나타낸 클래스 다이어그램이다.

테스트 요청 클래스는 사용자가 시스템에 테스트를 요청할 때 앱 아이디를 입력받아 전달하는 인터페이스이다.

정적 검증 수행 클래스는 전달받은 앱 아이디로 소스 코드, 배포 파일 및 IaC 코드 목록들을 조회한 후 이를 바탕으로 정적 검증을 수행한다. 인수 테스트 클래스는 전달받은 앱 아이디로 요구사항 명세 목록을 조회한 후, 이를 바탕으로 테스트 케이스를 생성하고 테스트 환경을 구축한 다음, 전달받은 소스 코드 및 배포 파일 목록들을 테스트 환경에 배포하여 인수 테스트를 실행하고 테스트 결과를 수집하여 반환하는 로직을 수행한다.

● 산출물 피드백

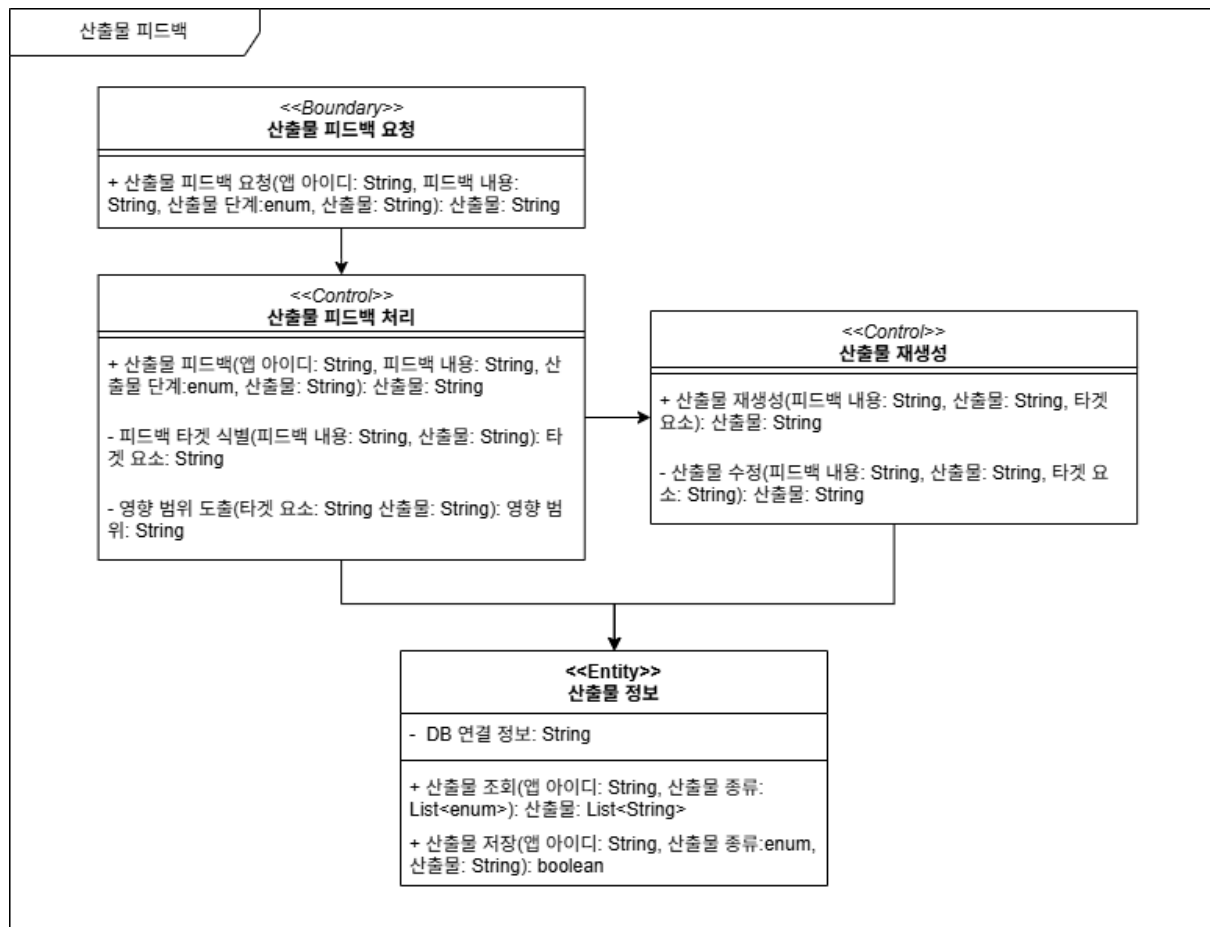


그림 7. 산출물 피드백 클래스 다이어그램

그림 7은 산출물 피드백에 필요한 구성 요소를 나타낸 클래스 다이어그램이다.

산출물 피드백 요청 클래스는 사용자가 산출물에 대한 피드백 반응을 요청할 때 앱 아이디, 피드백 내용, 대상 산출물 단계와 산출물을 입력받아 전달하는 인터페이스이다.

산출물 피드백 처리 클래스는 전달받은 정보를 바탕으로 대상 산출물에 피드백 내용을 반영하고자 할 때, 그 타겟이 되는 요소와 영향 범위를 도출한 후 해당 정보를 산출물 재생성 클래스에 전달한다. 산출물 재생성 클래스는 전달받은 내용을 바탕으로 산출물을 수정하고 DBMS에 저장한 후 반환하는 로직을 수행한다.

② UML 시퀀스 다이어그램

● 앱 아이디 발급

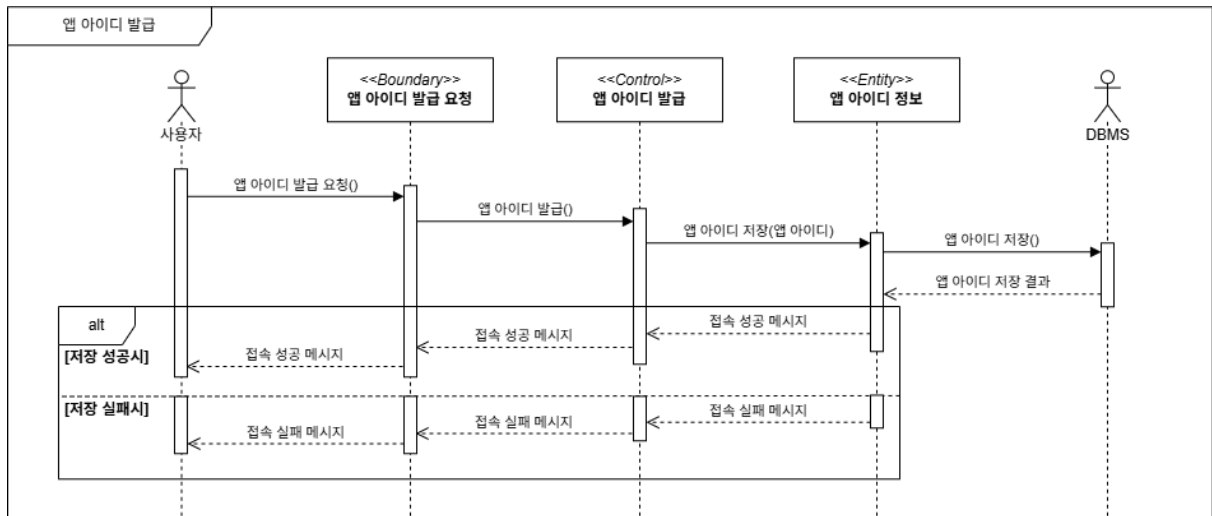


그림 8. 앱 아이디 발급 시퀀스 다이어그램

그림 8은 사용자의 요청에 따라 앱 아이디를 발급하고 저장하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 시스템에 앱 아이디 발급을 요청한다. 시스템은 새로운 앱 아이디를 발급한 뒤, 해당 앱 아이디를 DBMS에 저장을 시도하고 그 결과를 확인한다. 저장 성공 시 시스템은 사용자에게 접속 성공 메시지를 반환한다. 반면 저장에 실패할 경우에는 사용자에게 접속 오류 메시지를 반환한다.

● 요구사항 분석

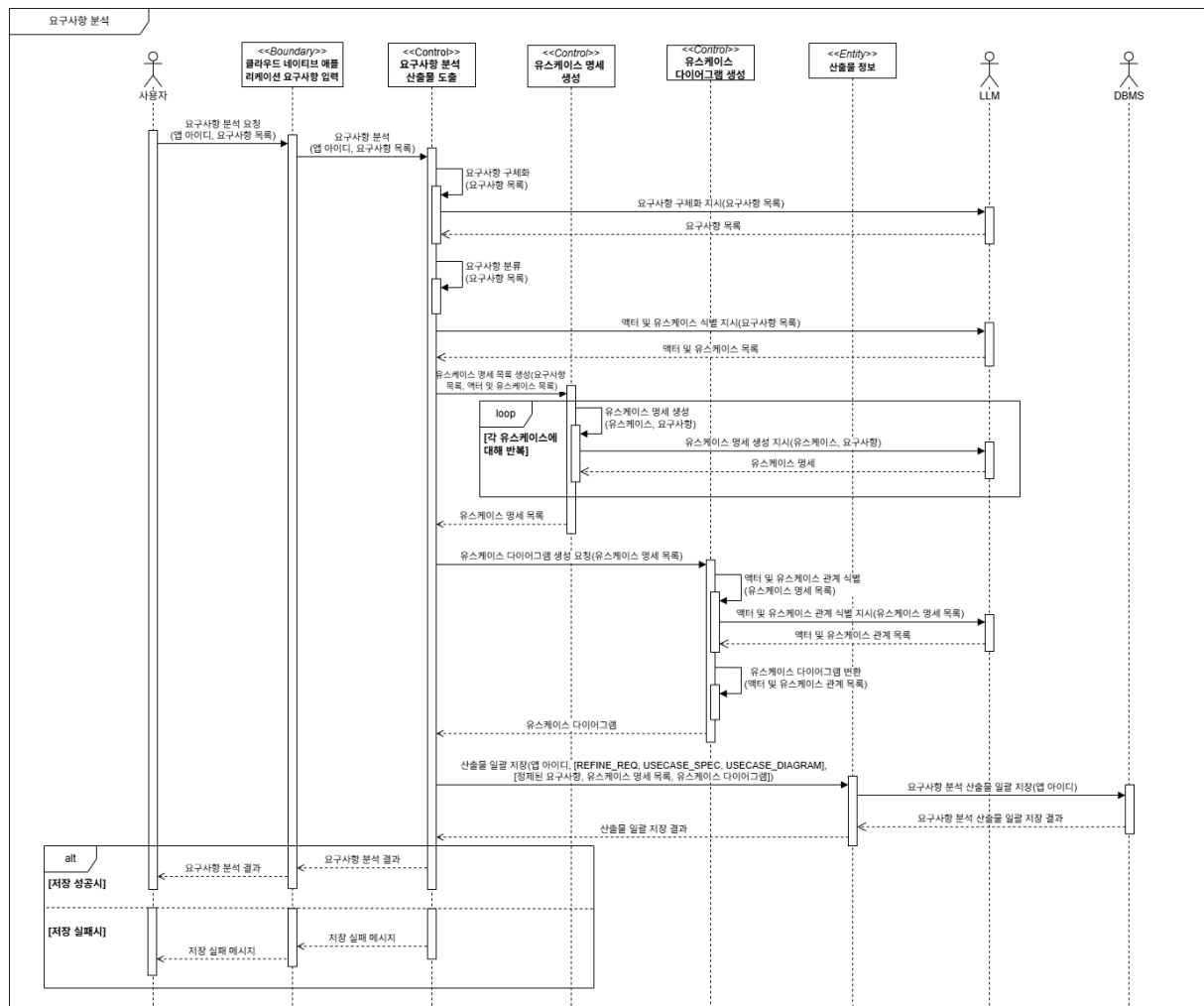


그림 9. 요구사항 분석 시퀀스 다이어그램

그림 9 는 사용자가 입력한 클라우드 네이티브 애플리케이션의 요구사항을 분석하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 클라우드 네이티브 애플리케이션 요구사항 목록을 전달하여 요구사항 분석을 요청한다. 요구사항 분석 산출물 도출 클래스는 전달받은 요구사항 목록을 구체화하고 분류 작업을 수행한 후 유스케이스 명세 목록 생성 클래스에 전달한다. 유스케이스 명세 목록 생성 클래스는 전달받은 요구사항 목록과 액터 및 유스케이스 목록을 바탕으로 유스케이스 명세 목록을 생성한 뒤 반환한다. 이후 요구사항 분석 산출물 도출 클래스는 반환받은 유스케이스 명세 목록을 유스케이스 다이어그램 생성 클래스에 전달한다. 유스케이스 다이어그램 생성 클래스는 유스케이스 명세 목록으로부터 액터 및 유스케이스 관계를 식별하고 유스케이스 다이어그램으로 변환한 후 반환한다. 요구사항 분석 산출물 도출 클래스는 도출된 요구사항 분석 산출물들을 산출물 정보 클래스를 통해 DBMS 에 일괄 저장을 시도하며, 저장 성공 시 요구사항 분석 결과를 사용자에게 반환한다. 저장 실패 시에는 저장 실패 메시지를 반환한다.

● 클라우드 리소스 명세 생성

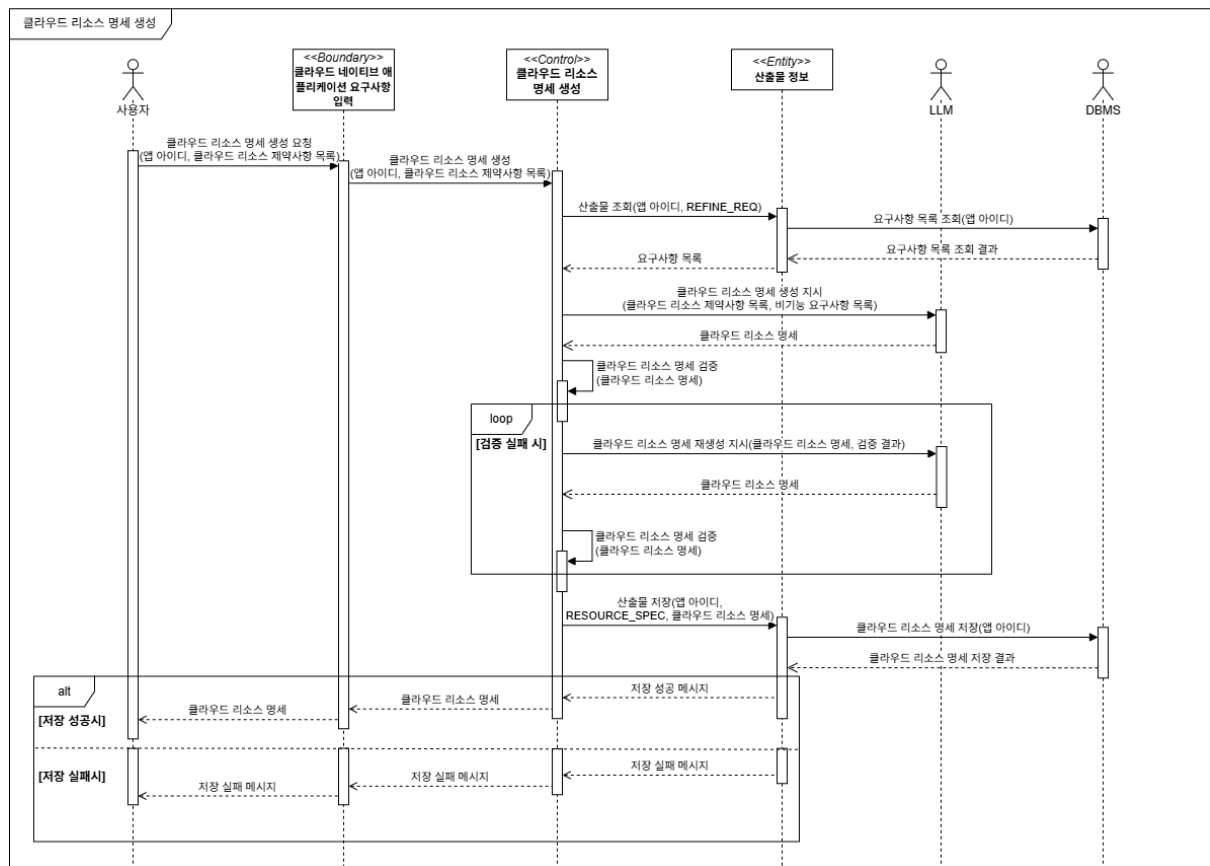


그림 10. 클라우드 리소스 명세 생성 시퀀스 다이어그램

그림 10 은 사용자가 입력한 클라우드 리소스 제약사항 목록을 바탕으로 클라우드 리소스 명세를 생성하는 시퀀스 다이어그램이다.

사용자는 앱 아이디와 클라우드 리소스 제약사항 목록을 전달하여 클라우드 리소스 명세 생성을 요청한다. 클라우드 리소스 명세 생성 클래스는 전달받은 앱 아이디로 요구사항 목록을 조회한 뒤, 그 중 비기능적 요구사항 목록과 클라우드 리소스 제약사항 목록으로부터 클라우드 리소스 명세를 생성하며, 해당 명세가 클라우드 네이티브 애플리케이션을 구축하기 위해 필요한 요소가 누락되지 않는지, 또는 사용자가 제시한 비용이나 리전 등의 제약 조건 하에서 실현 가능한지 판단하는 등 검증을 거친 후 클라우드 리소스 명세를 생성한다. 산출물 정보 클래스는 생성된 클라우드 리소스 명세를 DBMS 에 저장을 시도하며, 저장 성공 시 클라우드 리소스 명세를 사용자에게 반환한다. 저장 실패 시에는 저장 실패 메시지를 반환한다.

● 클래스 다이어그램 도출

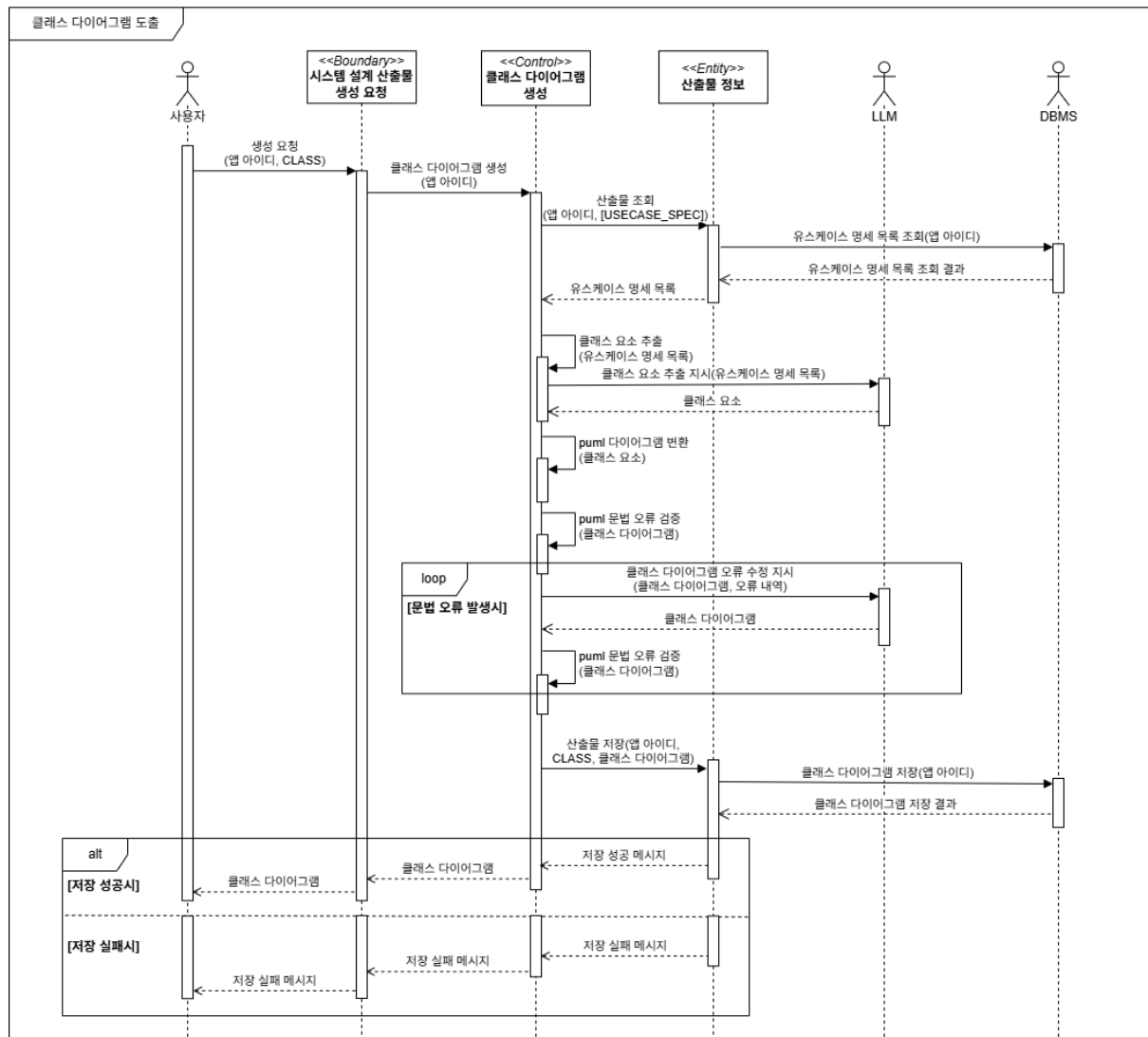


그림 11. 클래스 다이어그램 도출 시퀀스 다이어그램

그림 11 은 유스케이스 명세를 바탕으로 클래스 다이어그램을 도출하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디와 함께 클래스 다이어그램 생성을 요청한다. 시스템은 DBMS 에서 해당 앱 아이디의 유스케이스 명세를 조회한 뒤, LLM 을 활용하여 명세로부터 클래스 요소를 추출한다. 이후 시스템은 추출된 요소를 puml 다이어그램으로 변환하고 자체적인 문법 오류 검증을 수행한다. 검증 과정에서 문법 오류가 발생할 경우, LLM 을 통해 다이어그램 오류 수정을 지시하고 반환받아 재검증 과정을 거친다. 검증을 무사히 마친 시스템은 최종 생성된 클래스 다이어그램을 DBMS 에 저장을 시도하며, 저장 성공 시 완성된 클래스 다이어그램을 사용자에게 반환한다. 저장에 실패할 경우에는 저장 실패 메시지를 반환한다.

● 시퀀스 다이어그램 도출

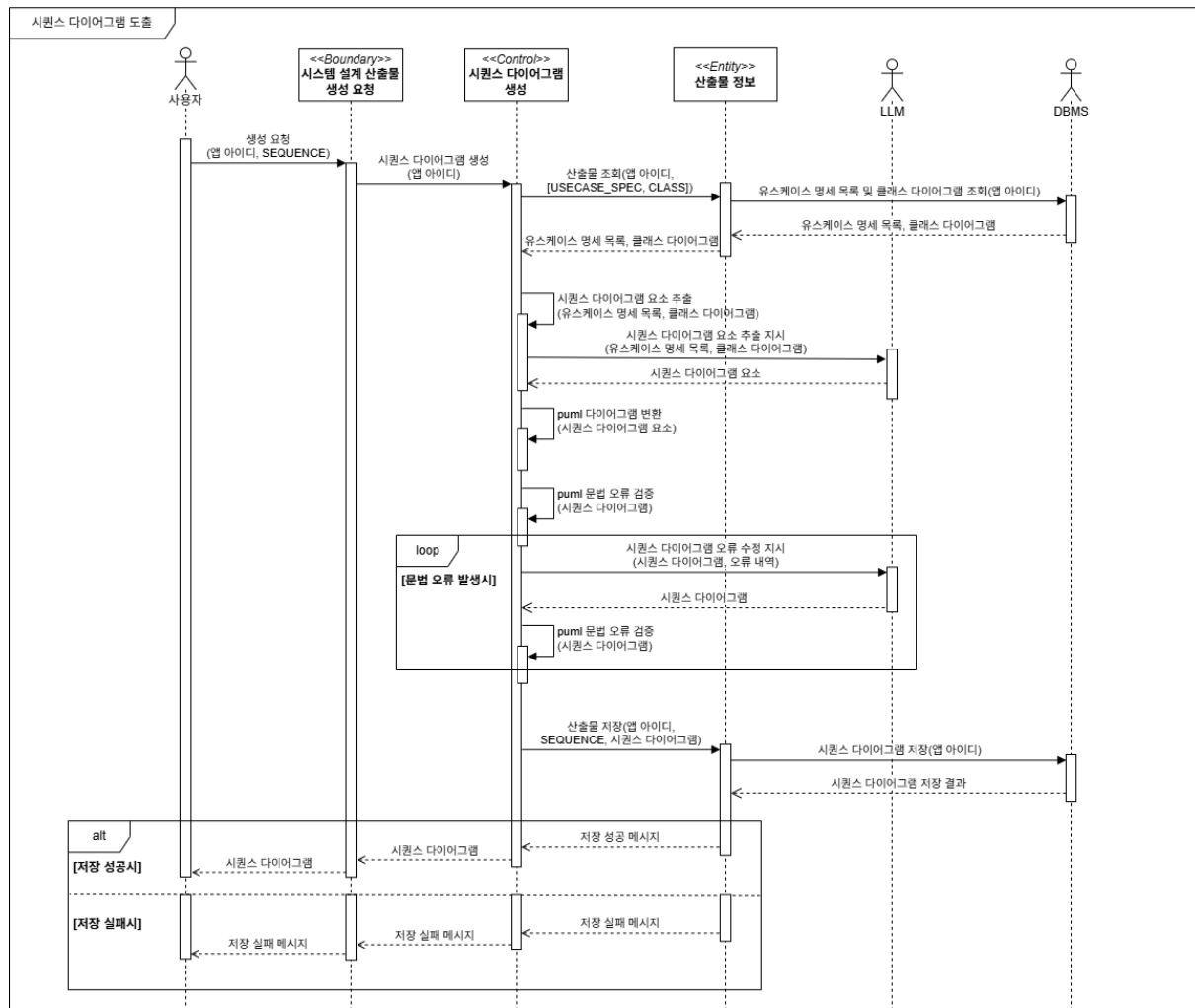


그림 12. 시퀀스 다이어그램 도출 시퀀스 다이어그램

그림 12는 유스케이스 명세와 클래스 다이어그램을 바탕으로 시퀀스 다이어그램을 도출하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디와 함께 시퀀스 다이어그램 생성을 요청한다. 시스템은 DBMS에서 해당 앱 아이디의 유스케이스 명세와 클래스 다이어그램을 조회한 뒤, LLM을 활용하여 해당 산출물들로부터 시퀀스 다이어그램 요소를 추출한다. 이후 시스템은 추출된 요소를 puml 다이어그램으로 변환하고 자체적인 문법 오류 검증을 수행한다. 검증 과정에서 문법 오류가 발생할 경우, LLM을 통해 다이어그램 오류 수정을 지시하고 반환받아 재검증 과정을 거친다. 검증을 무사히 마친 시스템은 최종 생성된 시퀀스 다이어그램을 DBMS에 저장을 시도하며, 저장 성공 시 완성된 시퀀스 다이어그램을 사용자에게 반환한다. 저장에 실패할 경우에는 저장 실패 메시지를 반환한다.

● API 명세서 도출

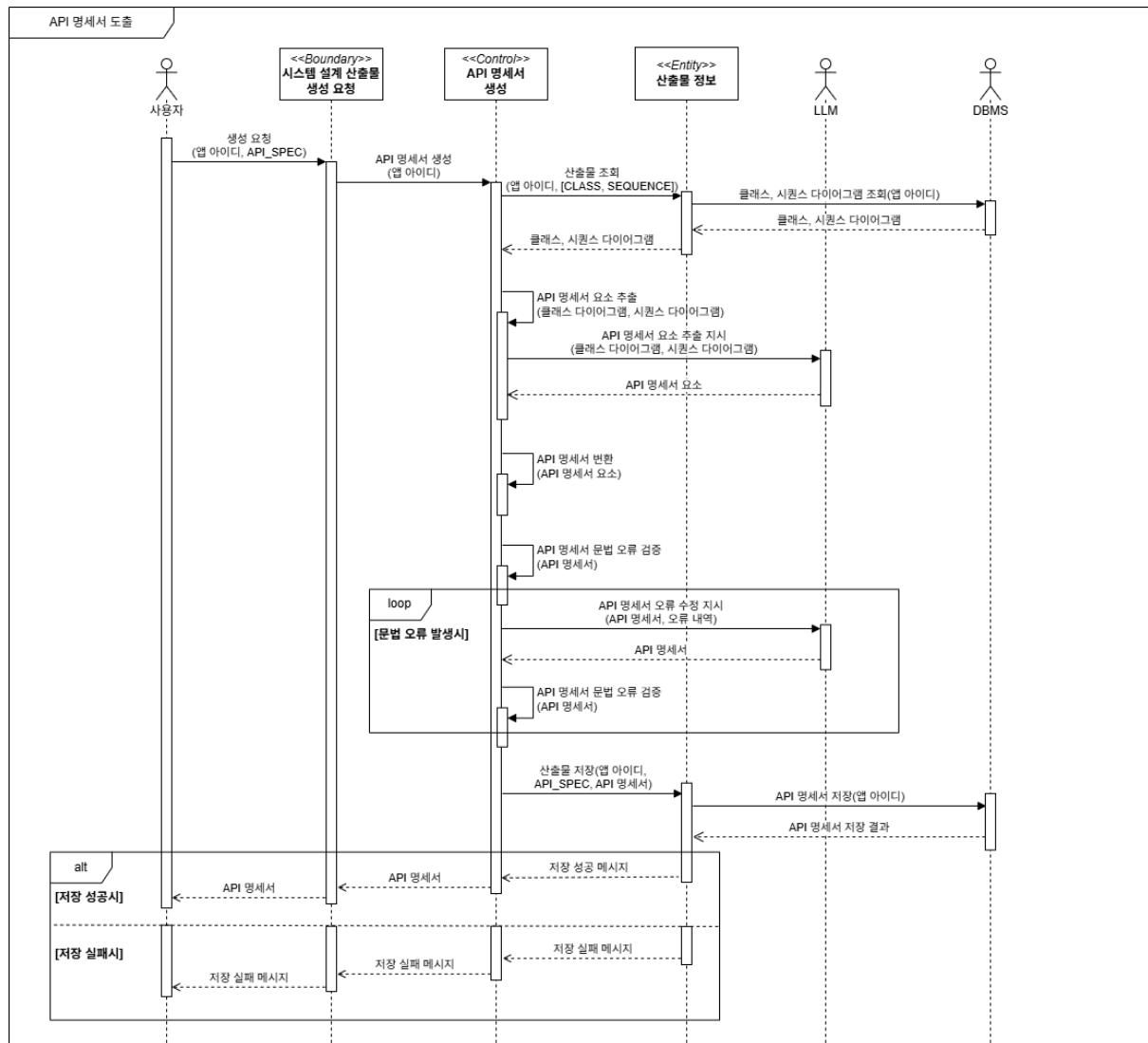


그림 13. API 명세서 도출 시퀀스 다이어그램

그림 13은 클래스 다이어그램과 시퀀스 다이어그램을 바탕으로 API 명세서를 도출하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디와 함께 API 명세서 생성을 요청한다. 시스템은 DBMS에서 해당 앱 아이디의 클래스 다이어그램과 시퀀스 다이어그램을 조회한 뒤, LLM을 활용하여 해당 산출물들로부터 API 명세서 요소를 추출한다. 이후 시스템은 추출된 요소를 API 명세서로 변환하고 자체적인 문법 오류 검증을 수행한다. 검증 과정에서 문법 오류가 발생할 경우, LLM을 통해 API 명세서 오류 수정을 지시하고 반환받아 재검증 과정을 거친다. 검증을 무사히 마친 시스템은 최종 생성된 API 명세서를 DBMS에 저장을 시도하며, 저장 성공 시 완성된 API 명세서를 사용자에게 반환한다. 저장에 실패할 경우에는 저장 실패 메시지를 반환한다.

● ER 다이어그램 도출

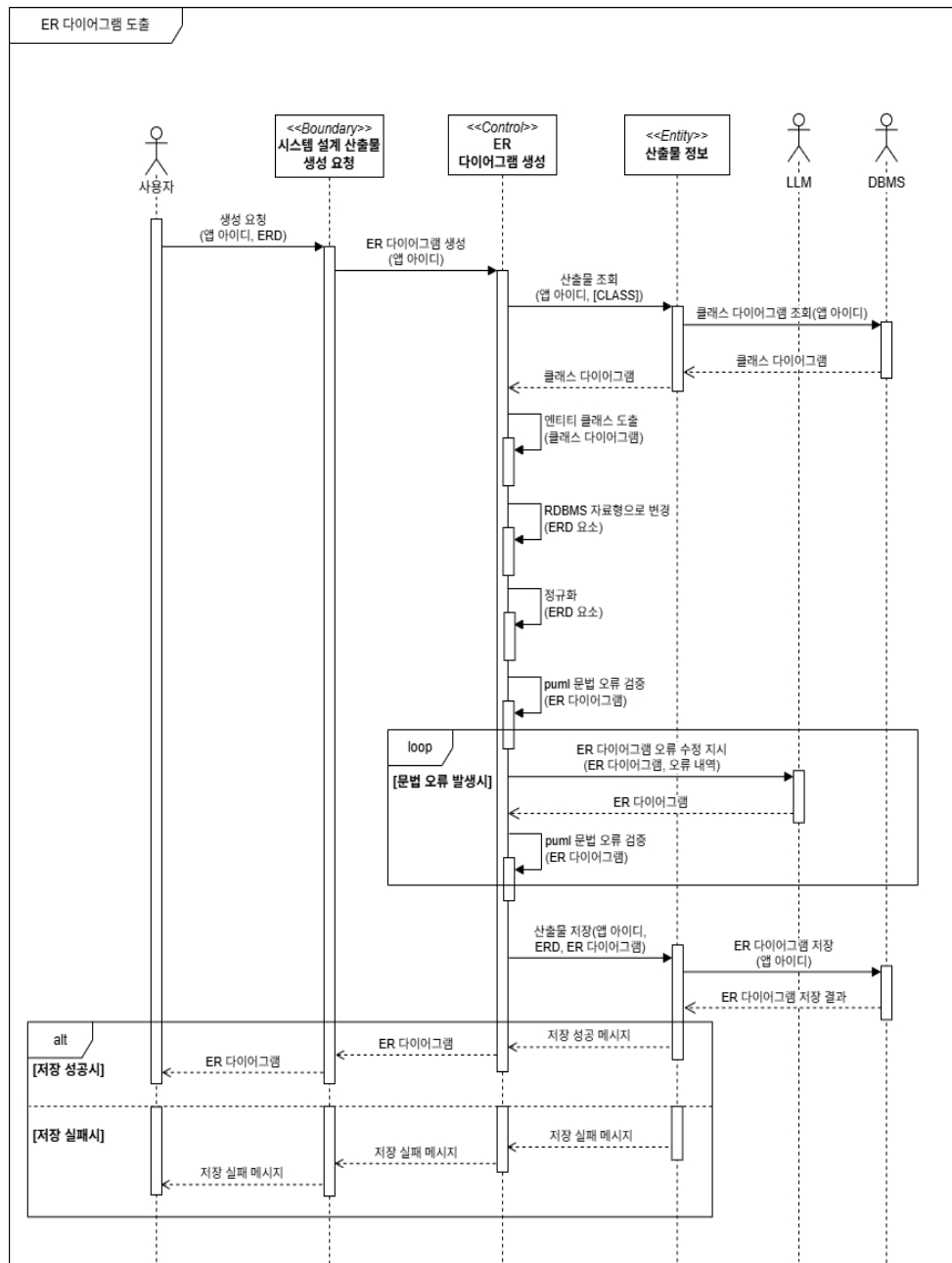


그림 14. ER 다이어그램 도출 시퀀스 다이어그램

그림 14 는 클래스 다이어그램을 바탕으로 ER 다이어그램을 도출하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디와 함께 ER 다이어그램 생성을 요청한다. 시스템은 DBMS 에서 해당 앱 아이디의 클래스 다이어그램을 조회한 뒤, 이를 바탕으로 엔티티 클래스를 도출한다. 이어서 도출된 ER 다이어그램 요소들을 RDBMS 자료형으로 변경하고 정규화 작업을 차례대로 수행한 후, 자체적인 puml 문법 오류 검증을 거친다. 모든 변환 및 검증 과정을 무사히 마친 시스템은 최종 생성된 ER 다이어그램을 DBMS 에 저장을 시도하며, 저장 성공 시 완성된 ER 다이어그램을 사용자에게 반환한다. 저장에 실패할 경우에는 저장 실패 메시지를 반환한다.

● 배포 다이어그램 도출

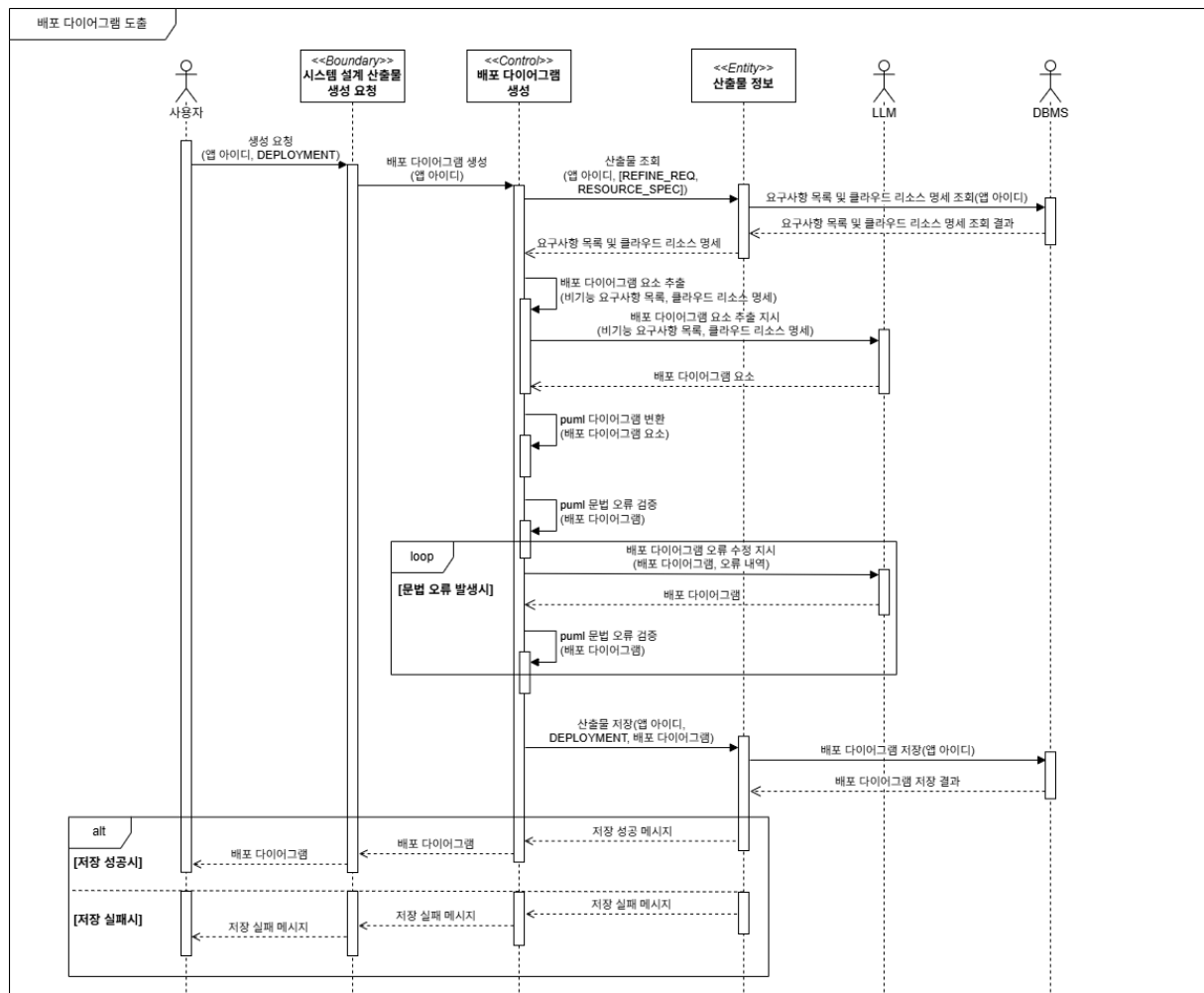


그림 15. 배포 다이어그램 도출 시퀀스 다이어그램

그림 15 는 비기능적 요구사항과 클라우드 리소스 명세를 바탕으로 배포 다이어그램을 도출하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디와 함께 배포 다이어그램 생성을 요청한다. 시스템은 DBMS 에서 해당 앱 아이디의 비기능적 요구사항과 클라우드 리소스 명세를 조회한 뒤, LLM 을 활용하여 해당 요구사항으로부터 배포 다이어그램 요소를 추출한다. 이후 시스템은 추출된 요소를 puml 다이어그램으로 변환하고 자체적인 문법 오류 검증을 수행한다. 검증 과정에서 문법 오류가 발생할 경우, LLM 을 통해 다이어그램 오류 수정을 지시하고 반환받아 재검증 과정을 거친다. 검증을 무사히 마친 시스템은 최종 생성된 배포 다이어그램을 DBMS 에 저장할 시도하며, 저장 성공 시 완성된 배포 다이어그램을 사용자에게 반환한다. 저장에 실패할 경우에는 저장 실패 메시지를 반환한다.

● 소스 코드 도출

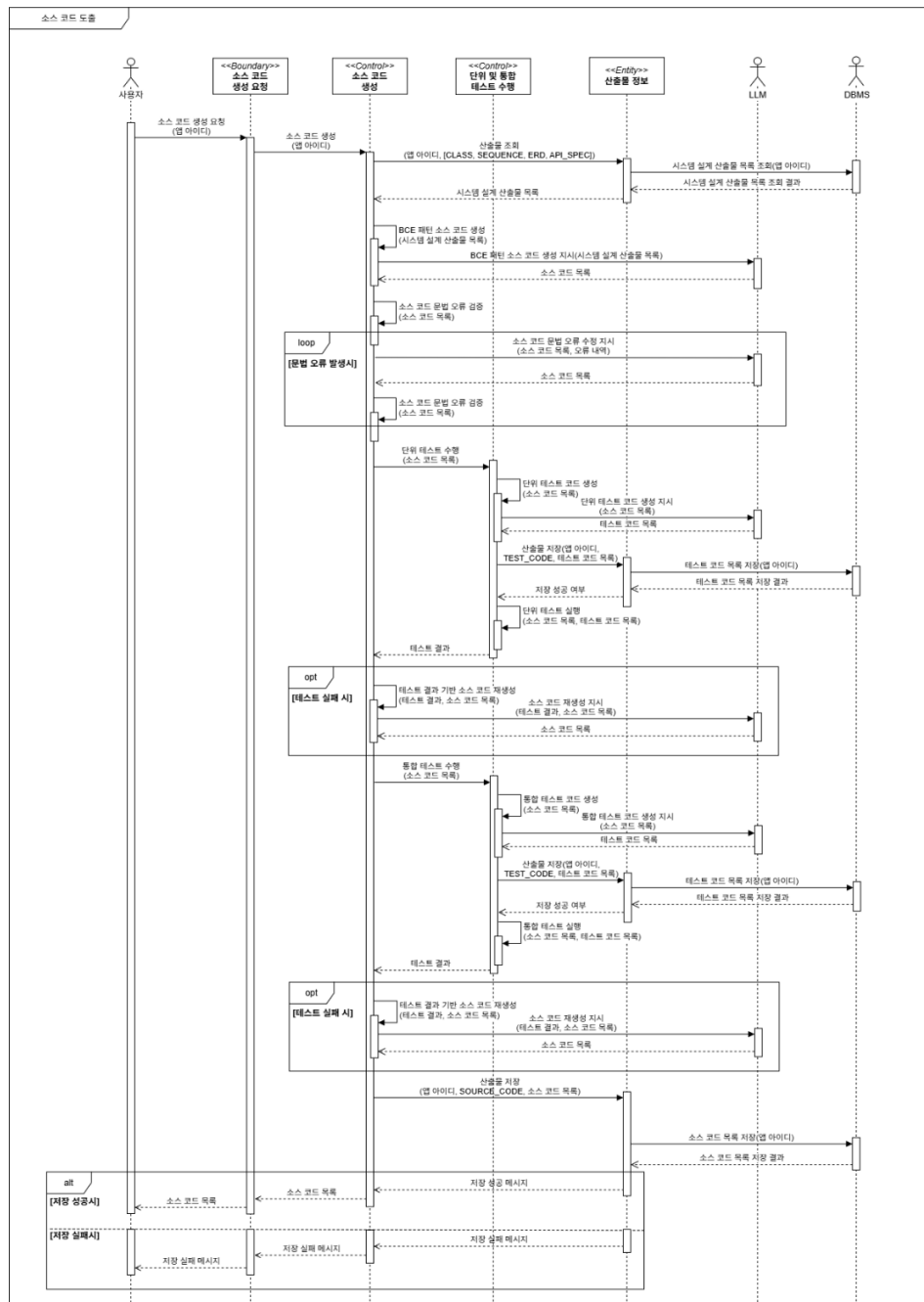


그림 16. 소스 코드 도출 시퀀스 다이어그램

그림 16 은 시스템 설계 산출물을 바탕으로 소스 코드를 도출하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디를 전달하여 소스 코드 생성을 요청한다. 시스템은 DBMS 에서 해당 앱 아이디의 시스템 설계 산출물 목록을 조회한 뒤, LLM 을 활용하여 BCE 패턴 소스 코드를 생성하고 소스 코드에 대한 자체적인 문법 오류 검증을 수행한다. 이후 단위 및 통합 테스트 를 수행한다. 모든 생성 및 검증 과정을 무사히 마친 후 시스템은 최종 소스 코드를 DBMS 에 저장을 시도하며, 저장 성공 시 완성된 소스 코드를 사용자에게 반환한다. 저장 실패 시에는 저장 실패 메시지를 반환한다.

● 배포 파일 도출

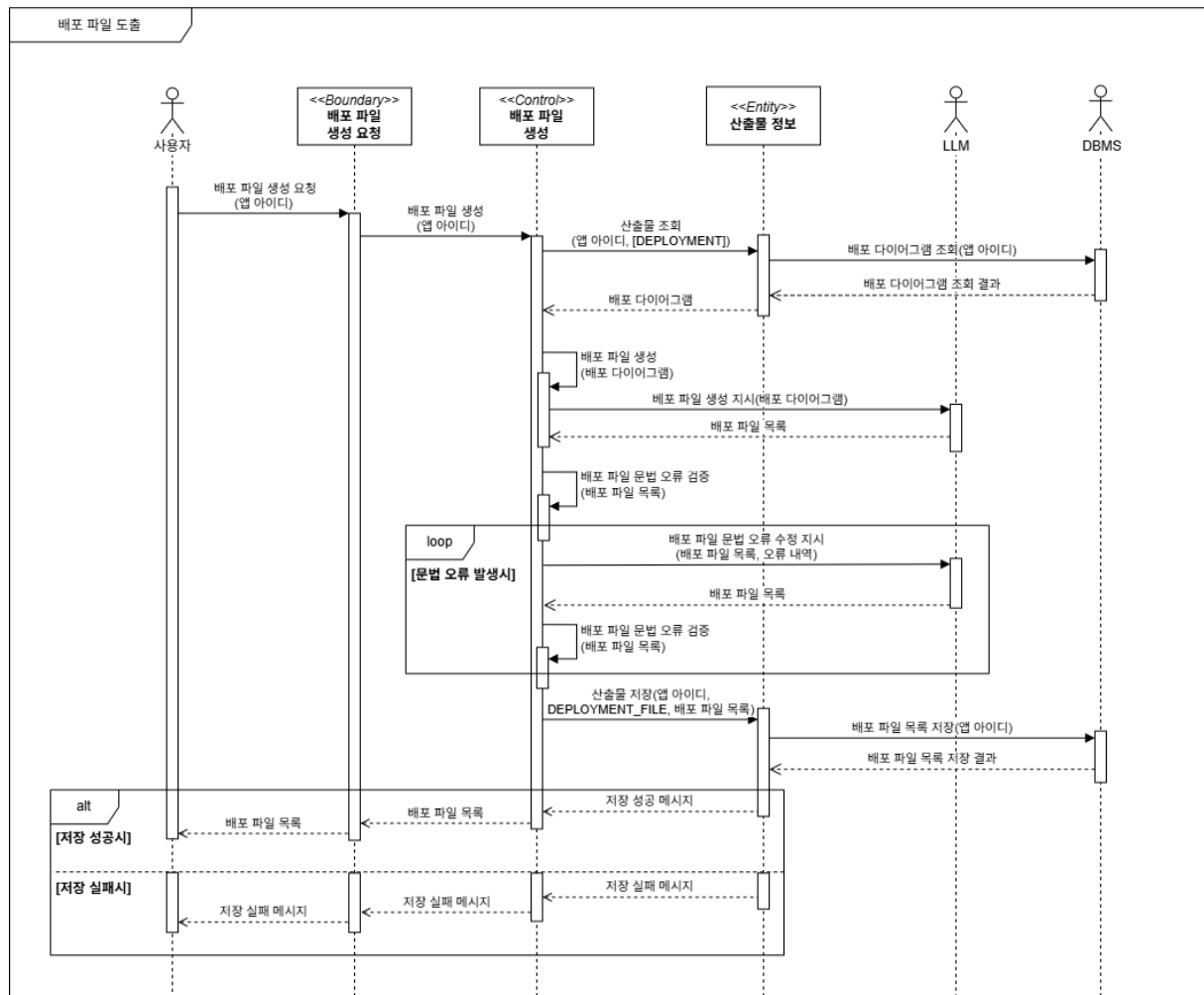


그림 17. 배포 파일 도출 시퀀스 다이어그램

그림 17은 배포 다이어그램을 바탕으로 배포 파일을 도출하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디를 전달하여 배포 파일 생성을 요청한다. 시스템은 DBMS에서 해당 앱 아이디의 배포 다이어그램을 조회한 뒤, LLM을 활용하여 해당 다이어그램으로부터 배포 파일을 생성하고 자체적인 문법 오류 검증을 수행한다. 생성 및 검증 과정을 무사히 마친 후 시스템은 최종 도출된 배포 파일을 DBMS에 저장을 시도하며, 저장 성공 시 완성된 배포 파일을 사용자에게 반환한다. 저장 실패 시에는 저장 실패 메시지를 반환한다.

● IaC 코드 도출

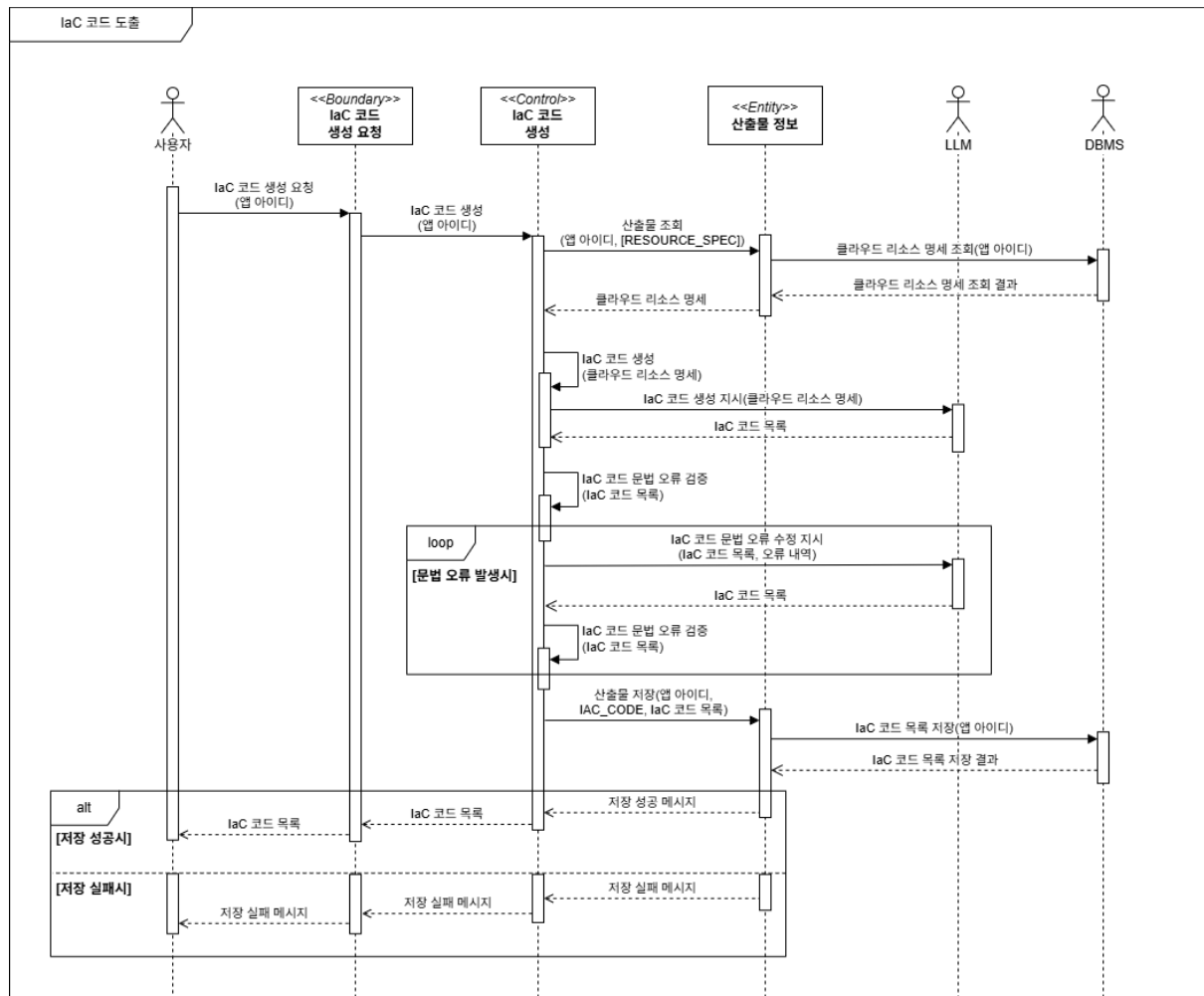


그림 18. IaC 코드 도출 시퀀스 다이어그램

그림 18 은 클라우드 리소스 명세를 바탕으로 IaC 코드를 도출하고 실제 테스트를 수행하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디를 전달하여 IaC 코드 생성을 요청한다. 시스템은 데이터베이스에서 해당 앱 아이디의 클라우드 리소스 명세를 조회한 뒤, LLM 을 활용하여 해당 명세로부터 IaC 코드를 생성하고 자체적인 문법 오류 검증을 수행한다. 생성 및 검증 과정을 무사히 마친 후 시스템은 최종 도출된 IaC 코드를 데이터베이스에 저장을 시도하며, 저장 성공 시 완성된 IaC 코드를 사용자에게 반환한다. 저장 실패 시에는 저장 실패 메시지를 반환한다.

● 테스트 산출물 도출

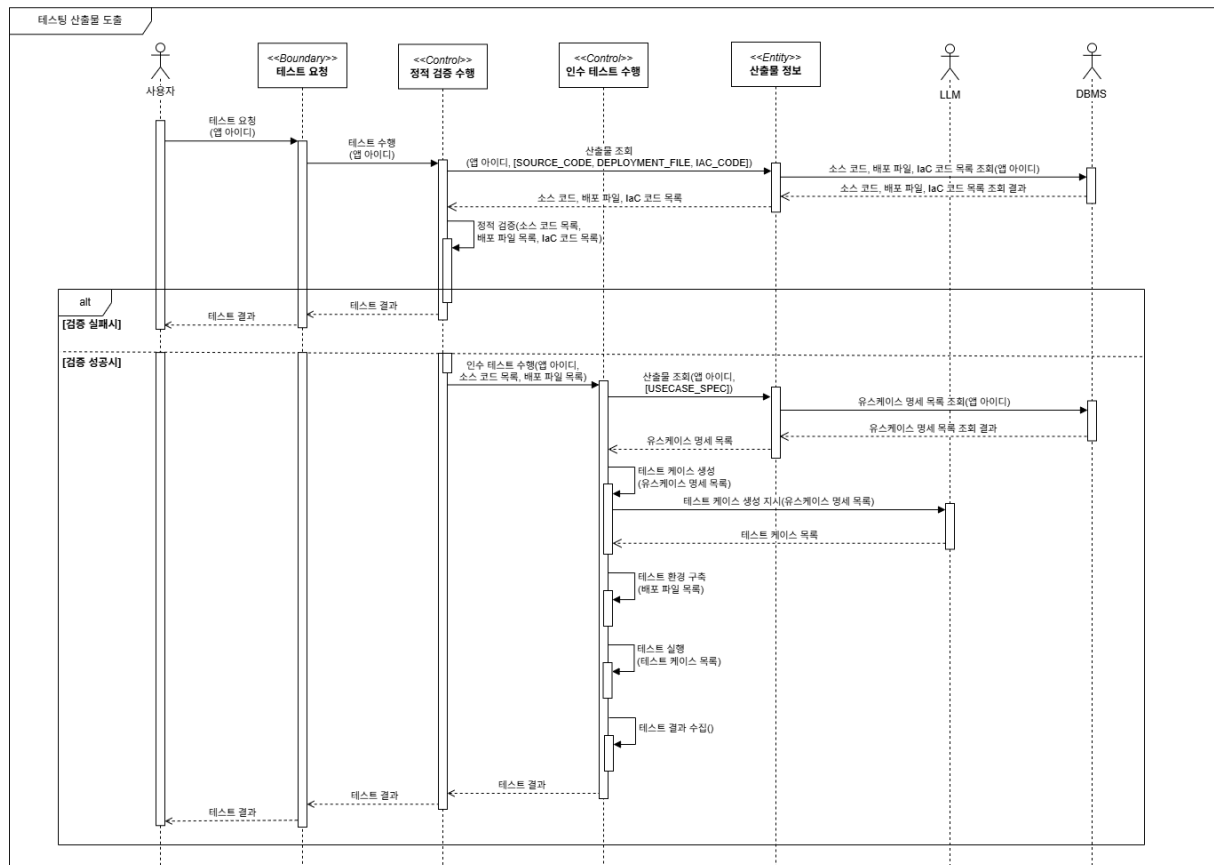


그림 19. 테스트 산출물 도출 시퀀스 다이어그램

그림 19 는 정적 검증을 수행하고, 유스케이스 명세 목록을 바탕으로 도출된 테스트 케이스를 통해 인수 테스트를 수행하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디를 전달하여 테스트를 요청한다. 시스템은 DBMS 에서 해당 앱 아이디를 기반으로 필요 산출물인 소스 코드, 배포 파일 및 IAC 코드 목록들을 조회한다. 해당 산출물을 바탕으로 정적 검증을 수행하고, 검증 실패 시 해당 결과를 반환한다. 검증 성공 시 인수 테스트에 필요한 산출물인 유스케이스 명세 목록을 조회한다. 조회된 산출물들을 바탕으로 LLM 에게 테스트 케이스 생성을 지시하여 테스트 케이스 목록을 반환받는다. 테스트 케이스가 준비되면 시스템은 내부적으로 테스트 환경을 구축하고, 앞서 생성된 테스트 케이스 목록을 기반으로 인수 테스트를 수행한다. 인수 테스트 수행이 완료되면 시스템은 테스트 도구로부터 테스트 결과를 수집하고, 도출된 테스트 결과를 사용자에게 반환한다.

● 산출물 피드백

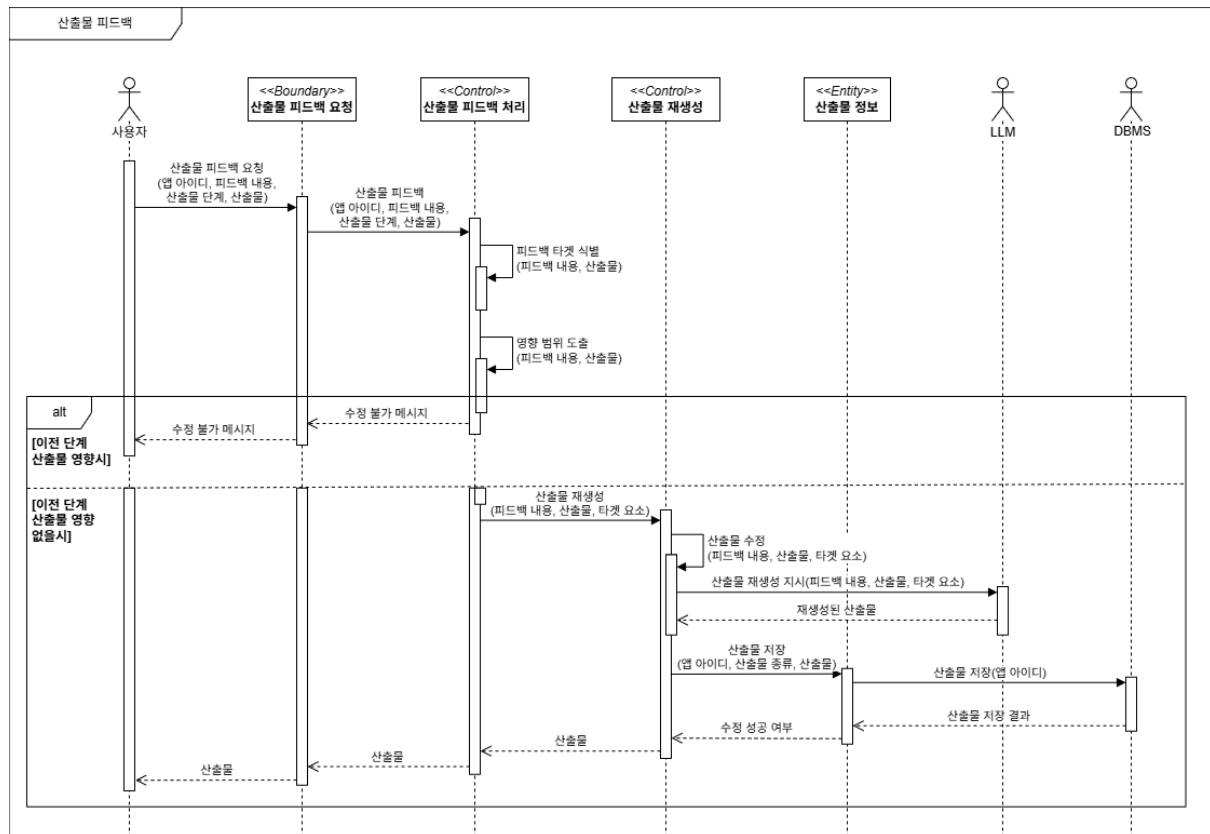


그림 20. 산출물 피드백 시퀀스 다이어그램

그림 20은 사용자의 피드백을 반영하여 기존 산출물을 수정 및 재생성하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디, 피드백 내용, 산출물 단계, 그리고 대상 산출물을 전달하여 시스템에 산출물 피드백을 요청한다. 요청을 전달받은 시스템은 내부적으로 피드백 타겟을 식별하고 산출물 수정에 따른 영향 범위를 도출한다. 이후 도출된 영향 범위에 따라 흐름이 분기되는데, 만약 이전 단계 산출물에 영향이 있을 경우 시스템은 사용자에게 수정 불가 메시지를 반환한다. 반면 이전 단계 산출물에 영향이 없을 경우, 시스템은 내부적으로 산출물 수정을 진행하며 LLM에게 산출물 재생성을 지시하여 피드백이 반영된 새로운 산출물을 반환받는다. 재생성된 산출물이 준비되면 시스템은 DBMS에 산출물 저장을 요청하여 수정 작업을 수행한다. DBMS 수정 작업이 완료되면, 시스템은 최종적으로 피드백이 반영된 완성된 산출물을 사용자에게 반환한다.

● 클라우드 네이티브 애플리케이션 다운로드

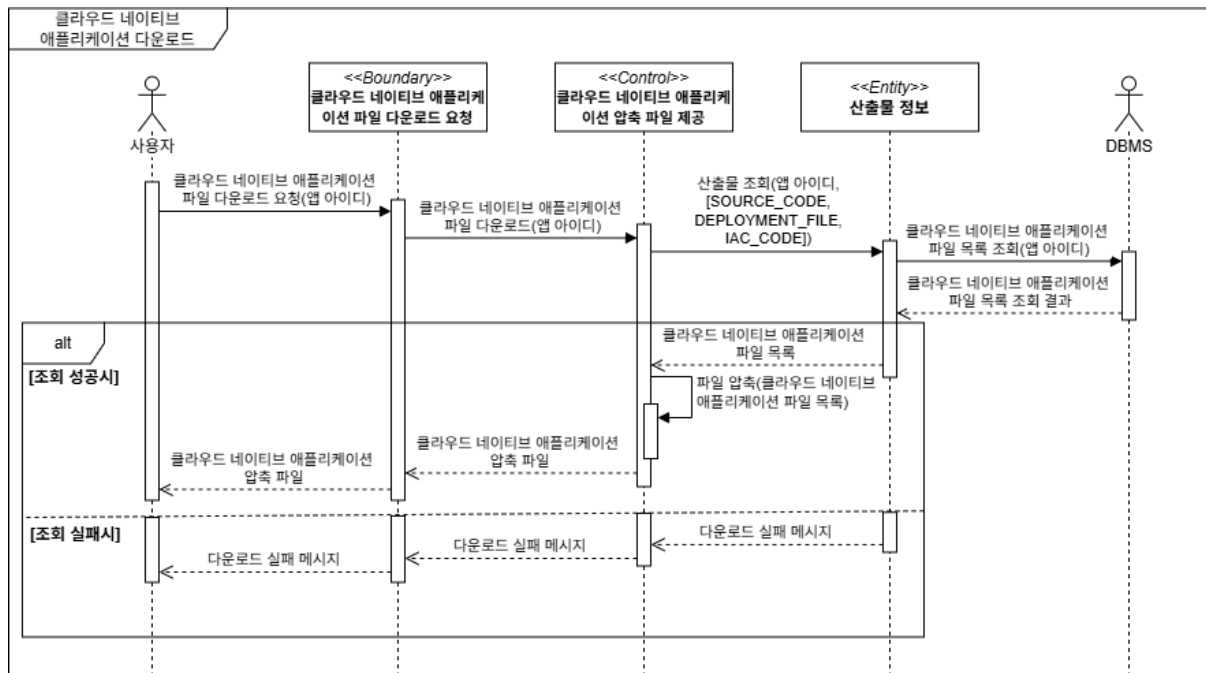


그림 21. 클라우드 네이티브 애플리케이션 다운로드 시퀀스 다이어그램

그림 21은 사용자의 요청에 따라 클라우드 네이티브 애플리케이션을 다운로드하는 과정을 나타낸 시퀀스 다이어그램이다.

사용자는 앱 아이디를 전달하여 시스템에 클라우드 네이티브 애플리케이션 파일 다운로드를 요청한다. 요청을 전달받은 시스템은 DBMS에서 해당 앱 아이디를 기반으로 시스템 구현 산출물인 소스 코드, 배포 파일 및 IaC 코드 목록에 대해 조회를 시도한다. 조회 실패 시, 시스템은 사용자에게 다운로드 실패 메시지를 전달한다. 조회 성공 시, 시스템은 해당 시스템 구현 산출물들을 단일 압축 파일로 패키징한 후 사용자에게 반환한다.

2) 전체 구조도

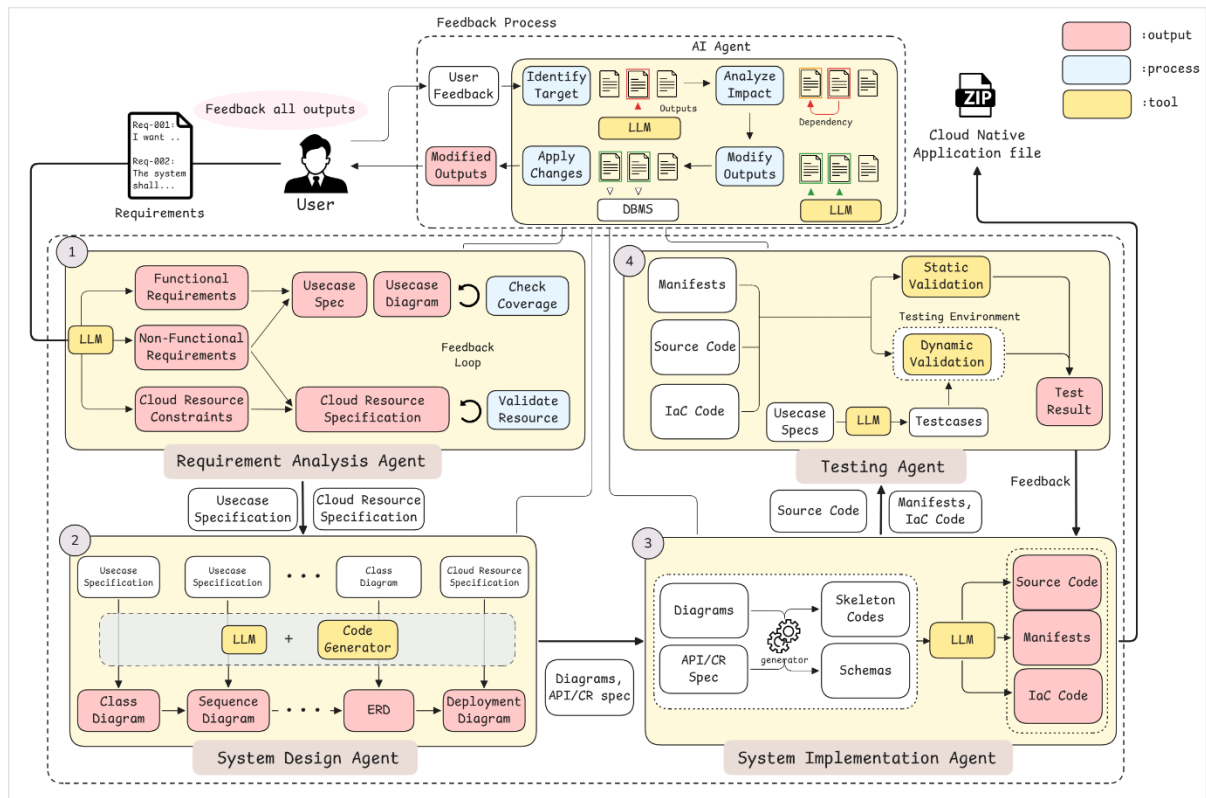


그림 22. 시스템 전체 구조도

그림 22는 시스템 전체 구조도를 나타낸다.

전체 시스템은 크게 요구사항을 분석하는 요구사항 분석 에이전트, 시스템 설계를 위한 시스템 아키텍처 다이어그램(클래스, 시퀀스, ER, 배포)과 API 명세를 생성하는 시스템 설계 에이전트, 소스 코드와 배포코드를 생성하는 시스템 구현 에이전트, 사용자의 기능적·비기능적 요구사항을 충족하는지를 테스트하는 테스트 에이전트로 구성된다. 이때, 개발 단계별 에이전트가 산출물을 생성할 때마다 사용자는 생성된 산출물에 대한 수정사항을 현재 진행중인 개발 단계의 에이전트에게 전달할 수 있으며, 해당 에이전트는 전달받은 사용자의 피드백을 기반으로 LLM을 통해 산출물을 수정할 수 있다. 또한 각 산출물에 대한 사용자의 피드백 요청이 있을 경우 다음과 같은 과정을 거친다. 우선 해당 피드백이 현재 산출물의 구성요소에서 어떤 요소를 수정하는가를 파악한다. 이후 해당 요소를 수정하게 될 때 어떤 범위의 내용까지를 수정해야 하는지 영향 범위를 파악한다. 마지막으로 파악된 영향 범위의 내용을 수정하여 사용자에게 반환한다. 각 개발 단계별 에이전트의 동작과정은 다음과 같다.

처음으로 요구사항 분석 에이전트는 사용자가 입력한 요구사항을 소프트웨어 요구사항 문서들로 학습된 LLM을 통해 구체화한다. 구체화된 요구사항은 기능적 요구사항과 비기능적 요구사항으로 분류된다. 유스케이스 작성 가이드라인을 학습한 LLM을 통해 분류된 기능적 요구사항으로부터 액터 및 유스케이스를 추출한 뒤 각 유스케이스와 기능적 요구사항을 토대로 유스케이스별 시나리오를 생성한다. 이렇게 생성된 시나리오와 비기능적 요구사항을 기반으로 유스케이스 명세가 작성되고, 시나리오를 통해 액터 및 유스케이스 간의 관계를 식별

한 후 해당 관계를 PlantUML 형식 변환 코드를 통해 유스케이스 다이어그램으로 변환한다. 또한, 클라우드 네이티브 제약사항은 비기능적 요구사항과 함께 클라우드 리소스 명세 생성 과정에 입력되며, 클라우드 네이티브 애플리케이션을 구축하기 위해 필요한 요소가 누락되지 않는지, 사용자가 제시한 비용이나 리전 등의 제약 조건 하에서 실현 가능한지 판단되는 등의 검증을 거친 후 조건에 부합하지 않는다면 사용자의 피드백에 따라 수정된다.

시스템 설계 에이전트는 이전 단계에서 생성된 유스케이스 명세의 시나리오에 기반하여 LLM 을 통해 클래스 요소를 추출한다. 추출된 클래스 요소는 PlantUML 의 작성 규칙에 따라 설계된 코드를 통해 PlantUML 다이어그램 코드로 변환되어 클래스 다이어그램으로 산출된다. 다음으로 생성된 클래스 다이어그램과 유스케이스 명세를 바탕으로 LLM 을 통해 시퀀스 다이어그램의 요소를 추출한다. 추출된 시퀀스 다이어그램 요소 역시 PlantUML 다이어그램 코드로 변환되는 과정을 거쳐 시퀀스 다이어그램으로 산출된다. 그 후, 완성된 시퀀스 다이어그램과 유스케이스 명세를 다시 LLM 의 입력물로 활용하여 API 설계에 필요한 요소들을 추출한다. 이 단계에서는 OpenAPI 생성기를 통해 표준화된 API 명세서를 생성하게 된다. 이어서 앞서 도출된 클래스 다이어그램으로부터 Entity 클래스를 추출하여 ER 다이어그램을 작성한다. 해당 단계는 LLM 을 거치지 않고 설계된 코드만으로 추출 및 변환을 수행한다. 또한, 제 1 정규화(1NF)를 적용하는 코드를 통해 ER 다이어그램 정규화를 수행한다. 마지막으로, 클라우드 리소스 명세를 기반으로 LLM 을 통해 인프라 및 배포 구조 요소를 추출한다. 추출된 요소는 최종적으로 PlantUML 다이어그램 코드로 변환되어 시스템의 물리적 구성도를 나타낸 배포 다이어그램을 생성하며 전체 설계 과정을 마무리한다.

시스템 구현 에이전트는 이전에 생성된 다이어그램들의 정보와 API 명세서를 기반으로 코드 생성 프레임워크를 통해 클래스 소스 코드의 뼈대와 DB 스키마를 작성한다. 이후 해당 산출물들을 바탕으로 오픈소스 AI 코딩 에이전트에 소스 코드, 배포 파일, 그리고 단위 및 통합 테스트 코드를 작성하는 작업을 할당한다. 할당된 작업이 완료되면 작성된 테스트 코드를 통해 단위 및 통합 테스트를 수행하며 테스트 결과에 따라 작성된 소스 코드를 수정하는 피드백 과정을 거친 후, 최종적으로 소스 코드 및 배포 파일을 산출한다.

테스팅 에이전트는 먼저 소스 코드, 배포 파일 및 IaC 코드에서 정적 검증을 수행한다. 검증 결과 이상이 없다면, 요구사항 명세 목록을 기반으로 LLM 을 통해 테스트 케이스를 생성한다. 이전에 산출된 소스 코드와 배포 파일을 별도로 격리된 테스트 환경에 배포한 후, 생성된 테스트 케이스와 테스트 시스템을 통해 테스트를 수행한다. 수행한 결과 통과하지 않은 테스트 케이스가 있거나 사전 검증에 이상이 있었을 경우 해당 결과를 시스템 구현 에이전트에게 전달하여 코드 수정 작업을 지시한다.

4. 갱신된 과제 추진 계획

과제 사전 조사 중, 요구사항 분석 및 시스템 설계 단계의 테스트 케이스 생성 및 실행 기능을 함께 구현하는 것보다 각 개발 단계별 산출물을 체계적으로 생성하고 사용자가 이를 직접 검토하여 의도를 반영하도록 하는 것이 과제 목표에 더 적합하다고 판단하였다. 이에 따라 테스트 케이스를 활용한 실행 기반 테스트는 실제 실행 대상과 관찰 가능한 결과가 존재하는 시스템 구현 이후 단계에서 수행하도록 설계를 수정하였다. 그러나 이 과정에서 구성원별 사전 지식 조사 기간이 증가하여 실제 개발에 활용할 수 있는 기간이 예상보다 단축되었고, 이에 따라 각 에이전트 개발과 백엔드/프론트엔드 개발을 병행하는 것으로 추진 계획을 변경하였다.

다음 표 3은 갱신된 과제 추진 계획을 나타낸다.

표 3. 갱신된 과제 추진 계획

개발구분	세부항목	5월		6월				7월				8월				9월		
		3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3
기획	주제 선정 및 고도화																	
	사전 조사																	
에이전트 개발	요구사항 분석 에이전트																	
	시스템 설계 에이전트																	
	시스템 구현 에이전트																	
	테스팅 에이전트																	
백엔드 및 프론트엔드 개발	배포 환경 구성																	
	데이터베이스 연동																	
	API 개발																	
	UI 개발																	
테스트	통합 테스트																	
	부하 및 복원력 테스트																	
유지보수	고도화 및 보완																	

5. 구성원 별 개발 진척도

표 4 는 구성원 별 개발 진척도이다.

표 4. 구성원 별 개발 진척도

이름	진척도
김재우	- 요구사항 분석 에이전트 개발 - 기능적/비기능적 요구사항 분류를 위한 BERT 모델 학습 및 테스트 - 요구사항으로부터 유스케이스 명세 및 다이어그램을 생성을 위한 LangGraph 기반 요구사항 분석 파이프라인 설계 - 유스케이스 명세 및 다이어그램 도출 기능 구현 - 테스팅 에이전트 개발 - Kubernetes 기반 애플리케이션 테스트 환경 구성 기능 개발 - 시스템 배포 환경 구성 - Azure ACR 및 AKS 기반 시스템 배포를 위한 리소스 프로비저닝 및 배포 스크립트 작성 및 테스트
이상현	- 요구사항 분석 에이전트 개발 - 요구사항 구체화 기능 구현 - 시스템 설계 에이전트 개발 - 유스케이스 명세로부터 시스템 설계 다이어그램(클래스, 시퀀스, ER, 배포)과 API 명세서 생성을 위한 LangGraph 기반 시스템 설계 파이프라인 설계 - 클래스 다이어그램 도출 기능 구현 - ER 다이어그램 도출 기능 구현 - 도출된 다이어그램의 PlantUML 문법 준수 여부 검증을 위한 렌더링 에러 로그 수집 기능 구현
황수환	- 요구사항 분석 에이전트 개발 - 유스케이스 다이어그램 도출 기능 구현 - 시스템 설계 에이전트 개발 - 시퀀스 다이어그램 도출 기능 구현 - API 명세서 도출 기능 구현

6. 과제 수행 내용 및 중간 결과

1) 요구사항 분석 에이전트

요구사항 분석 에이전트는 자연어로 입력된 클라우드 네이티브 애플리케이션 요구사항을 분석하여 유스케이스 명세 및 다이어그램을 생성하는 역할을 수행한다. 에이전트의 작업 과정은 사용자 요구사항 구체화 및 분류(기능적/비기능적 요구사항), 액터 및 유스케이스 식별, 유스케이스 명세 작성, 유스케이스 다이어그램 생성의 네 단계의 작업과 클라우드 리소스 명세를 생성하는 별도의 작업으로 구성된다. 그림 23은 요구사항 분석 에이전트의 작업 과정의 일부와 그 구현 내용을 나타낸다.

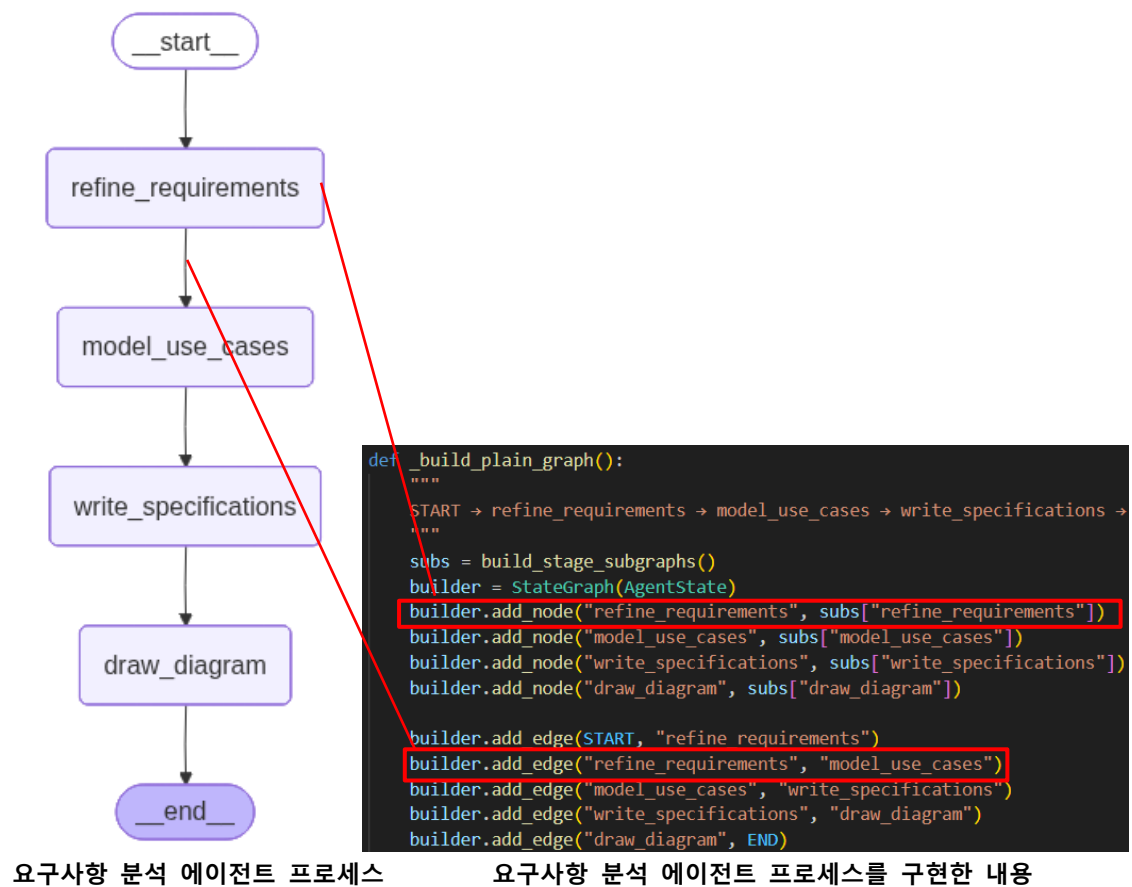


그림 23. 요구사항 분석 프로세스의 LangGraph 구조

일반적으로 사용자로부터 수집되는 초기 요구사항은 구체적인 기능이 드러나지 않는 추상적인 요구사항이 혼재되어 있을 가능성이 높으며, 이는 최종 결과물의 요구사항 불만족으로 이어질 수 있다. 이를 해결하기 위해 초기 요구사항을 구체화하는 과정이 필요하며, LLM과 Few-shot 프롬프팅을 도입하여 구현하였다. Few-shot 프롬프팅은 LLM에 여러 개의 예시를 전달하여 원하는 형태의 출력을 생성하는 방식이기 때문에, 과거 소프트웨어 개발에 사용되었던 요구사항 문서들로 구성된 PURE 데이터셋에서 요구사항 문장들을 추출하여 예시로 활용하였다. 또한 액터 및 유스케이스는 기능적 요구사항으로부터 도출할 수 있으므로, 구체화된 요구사항으로부터 기능적 요구사항을 분류할 방법이 필요하기에 자연어 문장 분류에 널리 쓰이는 BERT 모델을 파인 튜닝하여 분류기로 사용하였다. 파인 튜닝을 위한 학습 데이터는

앞서 언급한 PURE 데이터셋을 포함한 여러 요구사항 문서들에서 추출된 요구사항 문장들을 기능/비기능으로 태깅한 FR_NFR_Dataset 을 활용하였다. 그림 24 는 해당 데이터셋을 활용하여 BERT 모델을 파인튜닝하고 그 결과를 비교하여 사용할 BERT 모델을 결정한 과정을 나타낸다.

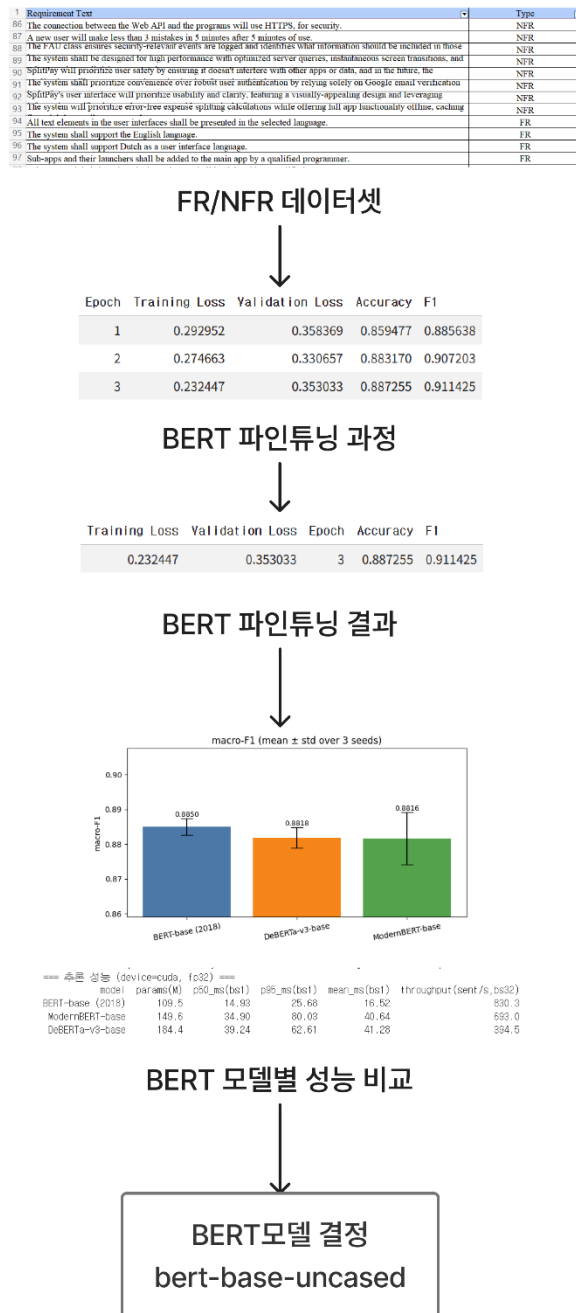


그림 24. 요구사항 문장 분류 BERT 모델 학습 및 결정 과정

분류된 기능적 요구사항을 바탕으로 액터와 유스케이스를 도출하는 단계와 유스케이스별 명세를 생성하는 단계에서는 LLM 의 자연어 이해 능력을 활용하되, 명확한 유스케이스 작성 방법론을 지시사항으로 전달하기 위해서 Alistair Cockburn 의 저서 《Writing Effective Use Cases》의 가이드라인을 참고하여 프롬프트를 설계하였다. 또한, 생성형 AI 모델의 환각 현상

을 방지하기 위해, 각 단계에서 산출된 결과에 대해 규칙 기반 검증과 LLM 기반의 의미론적 검증을 수행하는 과정을 추가하였다. 그림 25 는 유스케이스 명세를 생성하는 과정을 나타낸다.

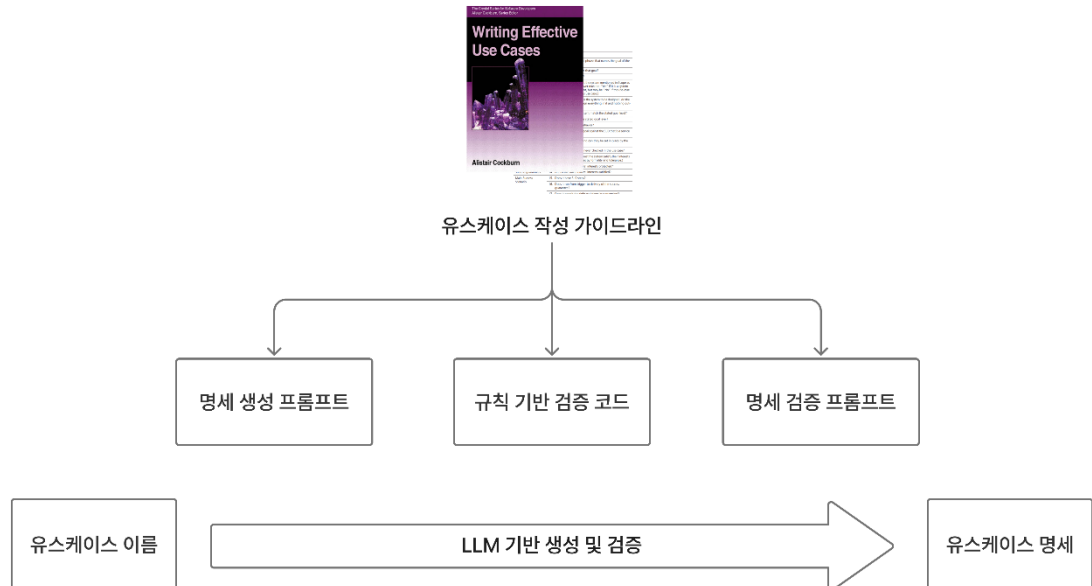


그림 25. 유스케이스 명세 생성 및 검증 과정

유스케이스 명세가 도출되면, 이를 기반으로 해당 유스케이스에 참여하는 액터와 명세에 포함된 시나리오를 기반으로 유스케이스 간 관계를 식별한 후, 식별된 관계를 바탕으로 코드를 통해 PlantUML 문법으로 변환하여 유스케이스 다이어그램을 산출한다.

마지막으로, 산출물에 대한 사용자의 피드백이 도착하는 경우 사용자의 피드백이 어느 작업에 해당하는지 결정한 후, 해당 작업에 사용자 피드백을 적용하여 이후 작업으로 진행하는 방식으로 산출물을 재생성한다.

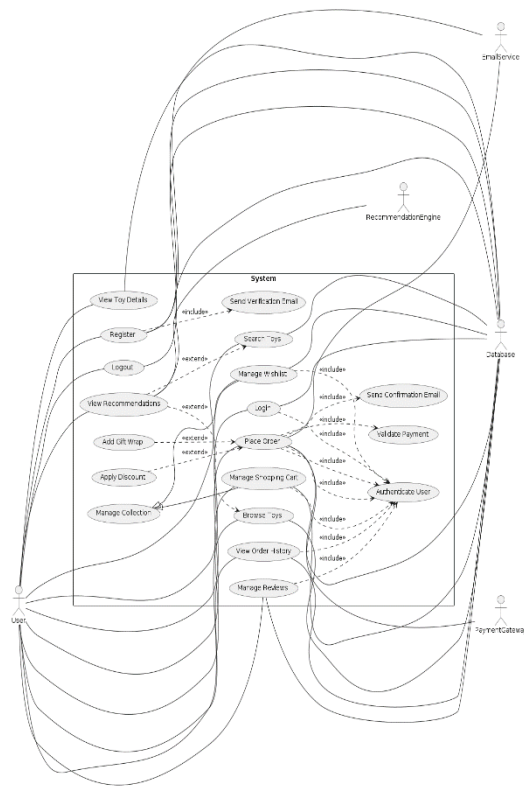
요구사항 분석을 단계별로 진행함으로써 개선되는 점을 파악하기 위해, 요구사항 분석 시위의 단계별 생성 과정 없이 바로 사용자 요구사항(기능적 요구사항 15 개, 비기능적 요구사항 8 개)을 LLM 에 전달한 결과(baseline)와 단계를 나누어 생성한 결과를 비교하는 실험을 진행하였다. 또한 단계를 나눈 경우 사용자 피드백을 주지 않았을 때와, 1 회 주었을 때를 비교하였다. 사용자 피드백은 산출물 생성 완료 시점에, 비기능적 요구사항을 유스케이스 시나리오에 포함시키지 말라는 취지의 자연어 문장으로 전달하였다. 다음 표 5 는 해당 실험 결과를 나타낸다.

표 5. 요구사항 분석 에이전트 산출물 평가 비교표

대상 \ 항목(개수)	baseline	사용자 피드백 0 회	사용자 피드백 1 회
액터	5	4	4
유스케이스	19	13	13

유스케이스 명세에 반영되지 않은 요구사항	7	0	0
유스케이스 시나리오에 포함된 비기능적 요구사항	0	3	0
LLM 기반 검증 재생성 횟수 초과 수(3 회)	- (검증 작업 없음)	6	1
include 관계	10	3	2
extend 관계	4	0	0
generalization 관계	2	1	1

그림 26은 baseline, 사용자 피드백 0 회, 사용자 피드백 1 회 각각에 대해 생성된 유스케이스 다이어그램을 나타낸다. 다이어그램 이미지는 모두 동일한 PlantUML 뷰어로 생성되었다.



baseline



사용자 피드백 0 회



사용자 피드백 1 회

그림 26. 각 케이스별 생성된 유스케이스 다이어그램

실험 결과, 단계별 생성 및 검증 과정이 존재할 경우 기능적 요구사항에 명시되지 않은 유스케이스 생성(환각)이 감소하였고, 유스케이스 명세의 요구사항의 반영률이 증가하였으며, 사용자의 피드백이 존재하는 경우 해당 피드백 내용이 결과물에 반영됨을 관찰할 수 있었다. 그러나, 사용자의 피드백이 의도하지 않은 결과(include 관계의 감소)도 관찰되었다. 따라서, 요구사항 분석을 단계별로 수행하며 검증 및 피드백을 적용하는 것이 바로 사용자 요구사항을 LLM에 전달하여 요구사항 분석 산출물을 생성하는 것보다 요구사항을 명확히 반영한다는 점과, 사용자의 의도를 더욱 정확히 반영할 수 있도록 피드백 회로를 개선할 여지가 있음을 알 수 있었다.

2) 시스템 설계 에이전트

시스템 설계 에이전트의 주요 역할은 생성된 유스케이스 명세의 시나리오를 기반으로 설계 산출물인 다이어그램(클래스 다이어그램, 시퀀스 다이어그램, ER 다이어그램, 배포 다이어그램)과 API 명세서를 도출하는 것이다. 해당 에이전트의 역할을 수행하기 위해 LangGraph 기반의 파이프라인을 구성하였으며 그림 27 은 시스템 설계 에이전트의 그래프 구조와 이를 구현한 코드에 대한 그림이다. 이때 node 는 실제 작업을 수행하는 기본 단위이며, edge 는 노드간의 연결과 실행흐름을 정의하는 요소이다.



시스템 설계 에이전트 프로세스

시스템 설계 에이전트 프로세스를 구현한 내용

그림 27. 시스템 설계 프로세스의 LangGraph 구조

그림 27 의 내용을 보면 클래스 다이어그램, 시퀀스 다이어그램, API 명세서, ER 다이어그램, 배포 다이어그램 순서로 생성하게 되는데, 산출물 도출 순서를 이렇게 설계한 이유는 다음과 같다.

유스케이스 시나리오를 수행하기 위해 필요한 주요 객체, 속성, 메서드를 클래스 다이어그램을 통해 우선적으로 도출한다. 이때 BCE 패턴을 적용하여 이후 Entity 를 이용해 ER 다이어그램을 도출 가능하도록 하고, Boundary 를 활용해 API 명세서를 도출 가능하도록 BCE 패턴을 적용한 클래스 다이어그램을 도출한다. 이렇게 도출된 내용을 바탕으로 참여객체를 클래스 다이어그램의 객체로 선택하여 유스케이스별 처리흐름을 시퀀스 다이어그램으로 구체화하고 클래스 다이어그램의 메서드가 부족하지 않은가를 확인하여 정합성을 검증한다. 이렇게 도출한 시퀀스 다이어그램으로부터 요청과 응답을 구체화하여 API 명세서를 도출한다. 이후 데이터베이스 설계를 위해 클래스 다이어그램의 Entity 클래스를 이용해서 ER 다이어그램을 도출

한다. 마지막으로 클라우드 네이티브 애플리케이션의 전체적인 자원배치 및 구조를 정하기 위한 배포 다이어그램을 생성한다.

이렇게 생성하는 5 가지의 산출물 생성 노드에는 각각 세부적인 흐름을 설계한 서브 그래프가 존재한다. 이는 공통적으로 LLM 이 생성한 결과물의 유효성을 스스로 검증하고 오류나 사용자의 피드백이 발생할 경우 스스로 재수정하는 루프를 포함한다. 이 중에 클래스 다이어그램 생성과 관련된 내용을 살펴보면 그림 28 은 클래스 다이어그램 생성 프로세스 노드의 서브 그래프 구조와 이를 구현한 코드에 대한 그림이다.

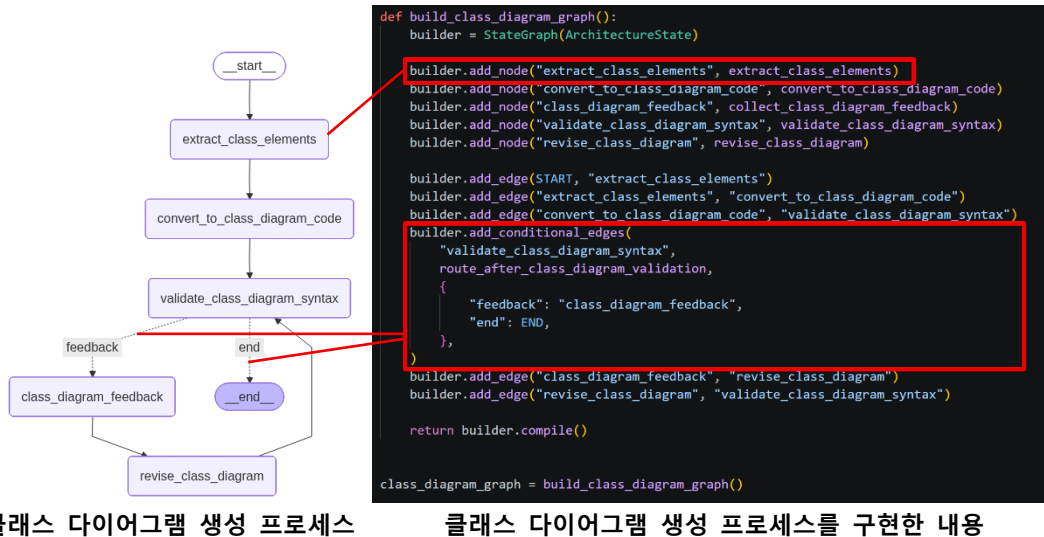


그림 28. 클래스 다이어그램 생성의 LangGraph 구조

위 그림의 내용처럼 서브 그래프는 클래스 요소 추출과 관련된 `extract_class_elements`, 추출된 요소를 diagram 으로 변환하는 `convert_to_class_diagram_code`, 생성한 다이어그램의 문법 오류를 검사하는 `validate_class_diagram_syntax`, 피드백이 있다면 진행하는 분기인 `class_diagram_feedback`, 클래스 다이어그램을 수정하는 `revise_class_diagram` 노드들로 구성되어있다.

분류된 기능적 요구사항을 바탕으로 클래스 요소를 추출하는 단계에서는 LLM 의 자연어 이해 능력을 활용하되, Jacobson 의 저서 《Object-Oriented Software Engineering: A Use Case Driven Approach》에서 유래한 BCE(Boundary-Control-Entity) 패턴이 실행마다 일관되게 반영되도록 규칙을 제약으로 명시하고, 명사-동사 분석에 기반한 도출 절차와 입출력 예시를 통해 암묵적 판단 기준을 함께 전달하는 방식으로 프롬프트를 설계하였다. 또한 생성형 AI 모델의 환각 현상을 방지하기 위해, 산출물이 반환되기 전 관계의 적합성과 통신 규칙 준수 여부를 스스로 재검토하는 자기검증 단계를 추가하였다.

프롬프트를 통해 클래스 요소가 도출되면, 이를 바탕으로 PlantUML 문법에 따라 변환하는 코드를 통해 클래스 다이어그램을 산출한다. 그러나, LLM 에게 PlantUML 형식 자체를 직접 생성하도록 요청할 경우 그림 29 와 같이 문법 오류로 다이어그램이 렌더링되지 않는 문제가 있었다.

```

PlantUML version 1.2026.7beta8 / efde38d [2026-07-08 21:04:53 UTC]
[From string (line 30) ]

@startuml
skinparam class {
  BackgroundColor<<boundary>> #F9F9F9
  BorderColor<<boundary>> #2A7FFF
  BackgroundColor<<control>> #FFF4E5
  BorderColor<<control>> #FF8C00
  BackgroundColor<<entity>> #E8F5E9
  BorderColor<<entity>> #2E7D32
  FontColor<<boundary>> #2A7FFF
  FontColor<<control>> #FF8C00
  FontColor<<entity>> #2E7D32
  ArrowColor #555555
  ArrowThickness 1
}

actor Subscriber
actor "Payment Gateway\n(external PG system)" as PG
actor "Customer Support Agent" as Support
actor "Billing/Settlement Team" as Billing

class SubscriptionManagementScreen <<boundary>> {
  Syntax Error? (Assumed diagram type: sequence)

```

그림 29. LLM 이 PlantUML 작성시 발생한 에러

이러한 문제를 해결하기 위해 LLM은 클래스 요소 추출까지만 담당하고 형식 변환은 별도로 작성한 코드가 처리하도록 하였으며, 출력 형식 오류로 인한 파이프라인 중단은 타입 힌트를 기반으로 데이터가 사전에 정의한 구조를 만족하는지 검증하는 데이터 검증 라이브러리 Pydantic을 활용하여 보완하였다. 그림 30은 클래스 다이어그램 요소로부터 PlantUML 형태로 변환하는 코드의 일부이다.

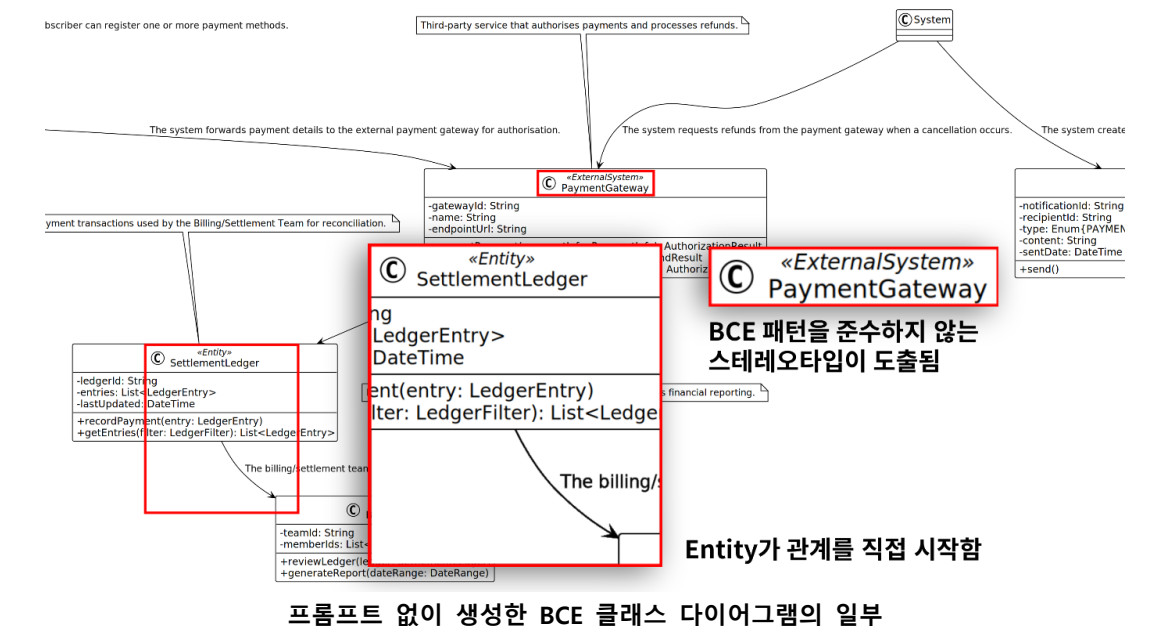


그림 30. 클래스 다이어그램 요소로부터 코드를 통해 PlantUML 형태로 변환

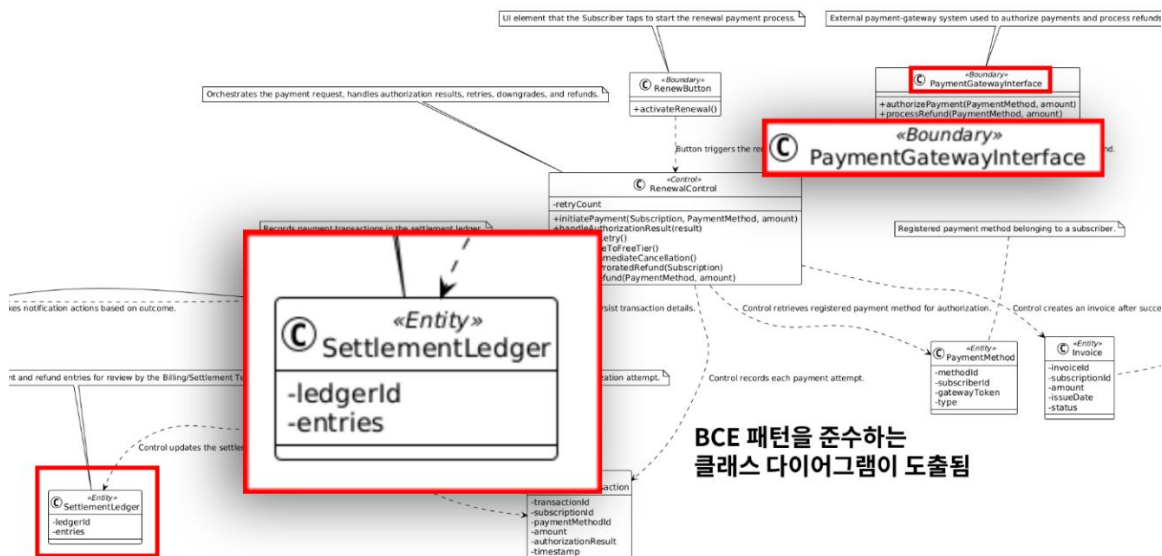
마지막으로, 산출물에 대한 사용자의 피드백이 도착하는 경우 사용자의 피드백이 어느 작업에 해당하는지 결정한 후, 해당 작업에 사용자 피드백을 적용하여 이후 작업으로 진행하는 방식으로 산출물을 재생성한다.

클래스 다이어그램 생성을 프롬프트의 단계별로 진행함으로써 개선되는 점을 파악하기 위해, 요구사항 분석 단계와 같이 단계별 생성 과정 없이 유스케이스 시나리오를 통해 바로 생성한

클래스 다이어그램과 단계를 나누어 생성한 결과를 비교하는 실험을 진행하였다.



프롬프트 없이 생성한 BCE 클래스 다이어그램의 일부



프롬프트를 통해 생성한 BCE 클래스 다이어그램의 일부

그림 31. 프롬프트의 유무에 따른 클래스 다이어그램 결과 비교

실험 결과, 프롬프트 적용 여부에 따른 산출 결과를 비교한 결과, 프롬프트 없이 생성한 다이어그램은 Boundary, Control 클래스가 도출되지 않거나 Entity가 관계를 직접 시작하는 등 BCE 통신 규칙이 지켜지지 않은 반면, 프롬프트를 적용한 결과는 모든 클래스가 세 스테레오타입 내에서 분류되고 Entity가 관계의 시작점이 되는 사례가 발견되지 않아, 설계한 프롬프트가 BCE 원칙 준수에 실질적으로 기여함을 확인하였다.

7. 향후 진행 계획

1) 시스템 구현 에이전트

시스템 구현 에이전트는 설계 단계에서 도출된 산출물을 기반으로 소스 코드와 배포 파일을 생성하는 역할을 담당한다. 관련 AI 에이전트 오픈소스 프로젝트를 검토한 결과, 소스 코드 작성뿐만 아니라 단위 및 통합 테스트까지 수행할 수 있는 OpenHands 를 도입하기로 결정하였다. 또한 해당 에이전트는 본 시스템의 주 개발 환경인 Python 기반의 소프트웨어 개발 키트(SDK)를 지원하므로 연동에 용이하다는 장점이 있다.

현재까지는 OpenHands 가 시스템 구현 과정에서 실행하는 터미널 명령이 본 시스템을 침범할 가능성을 막기 위해 격리된 실행 환경인 샌드박스를 구성하고, 기본적인 기능 검증을 완료하였다. 향후에는 에이전트가 복잡한 소스 코드 구조를 탐색하는 데 소요되는 비용을 절감하고 명확한 테스트 통과 기준을 제시할 수 있도록, 이전 개발 단계에서 생성된 설계 산출물로부터 스켈레톤 코드를 자동 생성하고 이를 시스템과 연동하는 기능을 추가로 개발할 예정이다.

2) 테스트 에이전트

테스팅 에이전트는 생성된 클라우드 네이티브 애플리케이션이 목표한 기능 및 비기능적 요구사항을 만족하는지 테스트하는 역할을 수행한다. 안전한 검증을 위해서는 테스트 대상 애플리케이션이 본 시스템에 영향을 주지 않도록 환경을 분리해야 하므로, 쿠버네티스 기반 격리 환경 생성 기능이 필요하다.

현재까지는 네임스페이스(Namespace) 기준의 격리까지만 구현되어 있으며, 외부의 공개 저장소에 이미 빌드된 컨테이너 이미지가 등록되어 있어야만 배포가 가능하다는 제약이 있다. 다만 부하를 주거나 장애를 주입하는 테스트를 진행할 경우 더 높은 수준의 격리가 필요할 것으로 예상되므로, 향후 격리 방법을 별개의 리소스를 할당하는 방식으로 확장할 수 있도록 구조를 설계하였다. 이와 함께 소스 코드를 컨테이너 이미지로 직접 빌드하여 배포하는 기능도 개발할 예정이다.

3) 시스템 배포 환경 구성

본 멀티 AI 에이전트 시스템의 비기능적 요구사항인 높은 가용성 및 안정적인 운용을 보장하고 장애 발생 시 신속한 복구를 달성하기 위해 컨테이너 오케스트레이션 플랫폼으로 쿠버네티스(Kubernetes)를 채택하였다. 시스템을 클라우드 환경에 배포하기 전 로컬 환경에서의 배포 테스트에는 Minikube 를 활용하고, 최종 배포 시에는 Azure 의 컨테이너 레지스트리인 ACR 및 쿠버네티스 서비스인 AKS 를 연동 및 활용할 예정이다.

현재까지는 클라우드 리소스가 할당된 상태만으로도 지속적인 비용이 발생한다는 문제를 해결하기 위해, 시스템 배포 테스트 수행 시점에만 자원을 동적으로 운용하는 PowerShell 스크립트를 작성하였다. 해당 스크립트는 리소스 프로비저닝, 이미지 빌드 및 배포, 리소스 삭제

2026 전기 졸업과제

과정을 자동으로 수행하여 불필요한 지출을 차단한다. 향후 본 시스템 뿐만 아니라 클라우드 네이티브 애플리케이션의 테스트 환경에서 요구하는 클라우드 리소스 또한 동일한 문제를 야기할 수 있으므로 위 스크립트의 역할을 테스트 에이전트의 작업 과정에 도입하는 등, 시스템이 동작하는 동안 발생할 수 있는 리소스에 의한 비용 낭비 문제를 줄일 수 있는 기능을 개발할 예정이다.